

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении научно-исследовательской работы
На тему: «Развёртывание кластера SLURM и генерация потока
пользовательских заданий»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Чуватов М. В.

«_____» _____ 2026г.

Реферат

Настоящий отчёт содержит 58 страниц, 11 рисунков, 4 таблицы и 4 приложения.

Отчёт посвящён развёртыванию вычислительного кластера под управлением SLURM и разработке программы генерации пользовательских заданий на основе статистики реальной очереди. В ходе работы была подготовлена среда из пяти виртуальных машин на базе openSUSE Leap: управляющий узел и четыре вычислительных узла. Для узлов настроены статическая адресация, разрешение имён, SSH-доступ по ключам, общая служба аутентификации MUNGE и конфигурация SLURM.

Для генерации потока заданий использован фрагмент набора данных SLURM, отфильтрованный по индивидуальному варианту: UID 50728 и имя задания `qe7`. По исходным данным рассчитаны распределения длительности выполнения и ошибки пользовательского прогноза времени. На основе полученных параметров реализован генератор пользовательских заявок SLURM, который формирует задания с командой `sleep`, задаёт плановое время выполнения и автоматически помещает задания в очередь. Результаты представлены через статистику выполненных заданий, графики исходных и сгенерированных распределений, а также сравнение планового и фактического времени выполнения.

Ключевые слова: SLURM, MUNGE, openSUSE Leap, кластер, виртуальная машина, пользовательская заявка, очередь заданий, логнормальное распределение, нормальное распределение, генерация заданий.

Содержание

Термины и определения	4
Сокращения	5
Введение	6
1 Постановка задачи	7
2 Основная часть	8
2.1 Подготовка виртуальной среды	8
2.2 Настройка сети и доступа	8
2.3 Установка MUNGE и SLURM	9
2.4 Обработка исходного набора данных	11
2.5 Аппроксимация исходных распределений	12
2.6 Генерация заданий	15
2.7 Запуск заданий и сбор фактической статистики	16
2.8 Анализ фактической ошибки прогноза	19
Заключение	22
Список использованных источников	24
Приложение А. Пример сгенерированного задания SLURM	26
Приложение Б. Код генератора пользовательских заявок	27
Приложение В. Конфигурация SLURM	31
Приложение Г. Код анализатора исходных и фактических данных, а также код точки запуска программы и анализатора	33

Термины и определения

В настоящем отчёте применены следующие термины с соответствующими определениями:

Вычислительный узел Узел кластера, на котором выполняются пользовательские задания.

Управляющий узел Узел кластера, на котором работает контроллер SLURM и выполняется управление очередью заданий.

Пользовательская заявка Файл задания SLURM с параметрами запуска и командой, которую необходимо выполнить на вычислительных узлах.

Манифест CSV-файл, в котором генератор сохраняет параметры сформированных заданий и расчётные временные характеристики.

Ошибка прогноза времени Запас времени между запрошенным или плановым временем выполнения задания и его длительностью.

Сокращения

В настоящем отчёте применены следующие сокращения:

SLURM Slurm Workload Manager, система диспетчеризации заданий и управления ресурсами вычислительного кластера. Исторически название SLURM расшифровывалось как Simple Linux Utility for Resource Management.

MUNGE MUNGE Uid 'N' Gid Emporium, служба аутентификации, используемая SLURM для проверки сообщений между узлами.

SSH Secure Shell, протокол защищённого удалённого доступа.

NAT Network Address Translation, механизм трансляции сетевых адресов.

UID User Identifier, числовой идентификатор пользователя в Unix-подобных системах.

UTC Coordinated Universal Time, всемирное координированное время.

IP Internet Protocol, сетевой протокол, используемый для адресации узлов в сети.

Введение

Суперкомпьютерные системы используются для выполнения вычислительных задач, требующих значительного объёма процессорного времени и памяти. Такие системы обычно имеют кластерную архитектуру: множество вычислительных узлов объединены сетью и управляются специализированной системой диспетчеризации заданий. Одной из наиболее распространённых систем такого класса является SLURM.

Качество планирования в вычислительном кластере зависит от того, насколько точно пользователь оценивает требуемое время выполнения задания. На практике пользователь часто указывает время с запасом: фактическое время выполнения оказывается меньше запрошенного. Из-за этого диспетчер вынужден резервировать ресурсы на большее время, чем действительно необходимо, что усложняет планирование очереди и может снижать загрузку узлов.

Цель работы — сгенерировать поток пользовательских заданий SLURM на основе статистики реальной очереди и проверить его выполнение в кластере. Для этого необходимо построить кластер из виртуальных машин, настроить сетевое взаимодействие между узлами, подготовить конфигурацию SLURM, проанализировать исходный набор данных, разработать генератор заданий и сопоставить расчётные параметры с фактическими результатами запуска.

В отчёте описаны этапы подготовки виртуальных машин, настройки сетевой связности и SSH-доступа, установки MUNGE и SLURM, выбора параметров конфигурации, обработки исходного набора данных, генерации заданий и анализа результатов выполнения.

1 Постановка задачи

Необходимо сгенерировать поток пользовательских заданий на основе статистики реальной очереди SLURM, подготовить кластер для запуска этих заданий и проанализировать полученные результаты выполнения.

Задачи.

1. Создать пять виртуальных машин: один управляющий узел и четыре вычислительных узла.
2. Настроить статическую адресацию, разрешение имён и сетевую связность между всеми узлами.
3. Настроить SSH-доступ по ключам между узлами без ввода пароля.
4. Установить и настроить MUNGE для единой аутентификации.
5. Установить SLURM, сформировать конфигурационный файл через официальный конфигуратор и разложить его на все узлы.
6. Проверить состояние вычислительных узлов средствами SLURM.
7. Отфильтровать исходный набор данных по индивидуальному варианту.
8. Рассчитать параметры распределений длительности выполнения и ошибки прогноза времени по исходному набору данных.
9. Реализовать генератор пользовательских заявок SLURM.
10. Запустить сгенерированные задания на кластере и собрать статистику выполнения.
11. Представить результаты выполнения и сравнить исходные, сгенерированные и фактические характеристики заданий.

В варианте работы обозначены: дистрибутив openSUSE Leap, UID 50728, имя задания `qe7`. Исходные записи набора данных фильтруются по указанному UID и имени задания.

Результатом работы должны стать настроенный кластер SLURM, исходный код генератора, набор сгенерированных заданий, результаты выполнения на кластере, сравнение расчётных и фактических характеристик и отчёт, оформленный в соответствии с ГОСТ 7.32–2017.

2 Основная часть

2.1 Подготовка виртуальной среды

Практическая часть работы выполнялась на локальной машине с использованием Oracle VM VirtualBox¹. В качестве модели вычислительного кластера была использована схема из пяти виртуальных машин: один управляющий узел и четыре вычислительных узла. Управляющий узел получил имя `mgmt`, вычислительные узлы получили имена `cn01`, `cn02`, `cn03` и `cn04`.

На все виртуальные машины был установлен дистрибутив `openSUSE Leap`². При установке была выбрана минимальная консольная конфигурация без графической среды.

Каждой виртуальной машине был выделен один виртуальный процессор и 2 ГБ оперативной памяти. Первоначально для системного диска рассматривался объём 8–10 ГБ, однако установщик `openSUSE Leap` требовал не менее 12,5 ГиБ под корневой раздел и дополнительно 1–2 ГиБ под раздел подкачки. После этого виртуальные машины были пересозданы с диском объёмом 16 ГБ.

В результате была подготовлена однородная виртуальная среда: все узлы используют один дистрибутив, одинаковые аппаратные ограничения и минимальный набор служб. Сначала установка и базовая настройка были выполнены на управляющем узле, после чего тот же порядок действий был повторён на вычислительных узлах.

2.2 Настройка сети и доступа

Для каждой виртуальной машины использовались два сетевых адаптера. Первый адаптер был оставлен в режиме NAT. Он нужен для доступа к внешним репозиториям `openSUSE` и установки пакетов через `zypper`³. Вторым адаптером был подключён к внутренней сети VirtualBox. Именно через этот адаптер организована локальная связь узлов кластера между собой.

Для внутренней сети был выбран диапазон IP-адресов `10.1.0.0/24`. Управляющему узлу назначен адрес `10.1.0.21`, вычислительным узлам назначены адреса `10.1.0.31`, `10.1.0.32`, `10.1.0.33` и `10.1.0.34`. Адреса

¹Руководство пользователя Oracle VM VirtualBox: <https://www.virtualbox.org/manual/>.

²Документация `openSUSE Leap`: <https://doc.opensuse.org/>.

³Справка по использованию `zypper`: https://en.opensuse.org/SDB:Zypper_usage.

управляющего и вычислительных узлов разнесены по разным десяткам, поэтому конфигурация остаётся читаемой и допускает добавление новых узлов без изменения выбранной схемы.

Для доступа к виртуальным машинам с хостовой системы в VirtualBox были настроены пробросы SSH-портов через NAT: для `mgmt` использовался порт 2201, для `cn01–cn04` — порты 2202–2205. Это позволяло подключаться к каждому узлу с локальной машины, не открывая внешний доступ к внутренней сети кластера.

После настройки проброса портов на локальной машине была создана SSH-пара ключей и подготовлен пользовательский конфигурационный файл `~/.ssh/config`. В нём для каждого узла были указаны локальный адрес, соответствующий проброшенный порт и приватный ключ. После этого подключение с хостовой системы выполнялось по коротким именам узлов.

Разрешение имён выполнено через файл `/etc/hosts`. На всех узлах были добавлены одинаковые записи для имён `mgmt`, `cn01`, `cn02`, `cn03` и `cn04`.

Далее был настроен SSH-доступ без ввода пароля. Для этого на каждом узле была создана SSH-пара ключей, публичные ключи всех узлов были добавлены в файл `/etc/authorized/keys` на каждом узле. Дополнительно был создан пользовательский SSH-конфиг, в котором имена узлов сопоставлены с соответствующими хостами и выбранным приватным ключом. После настройки команда вида `ssh cn01` выполняется с любого узла без указания ключа и без ввода пароля.

2.3 Установка MUNGE и SLURM

Для работы SLURM требуется единый механизм аутентификации сообщений между узлами. Эту функцию выполняет MUNGE. На управляющем узле был создан общий ключ MUNGE, после чего он был скопирован на все вычислительные узлы. Для ключа были выставлены ограниченные права доступа и владелец `munge`; одинаковый ключ на всех узлах является условием корректного обмена служебными сообщениями SLURM.

Для корректной работы MUNGE также требуется согласованное системное время на всех узлах. При первоначальной установке на вычислительных узлах остался часовой пояс UTC+2, а на управляющем узле был установлен UTC+3. Из-за расхождения времени вычислительные узлы не мог-

ли корректно подключиться к кластеру: MUNGE проверяет время создания служебных сообщений и отклоняет сообщения, которые заметно отличаются от локального времени узла. После установки одинакового часового пояса и проверки текущего времени на всех виртуальных машинах службы MUNGE и SLURM начали взаимодействовать корректно.

Пакеты устанавливались через пакетный менеджер `zypper`. На управляющем узле используется служба `slurmctld`, которая хранит состояние очереди, принимает задания и назначает ресурсы. На вычислительных узлах используется служба `slurmd`, которая сообщает управляющему узлу о состоянии узла и выполняет назначенные задания.

Конфигурация SLURM была сформирована через официальный конфигуратор SchedMD⁴. В конфигурации указан кластер `practice`, управляющий узел `mgmt`, вычислительные узлы `cn[01-04]` и очередь `debug`. Для выбора процесса отслеживания был оставлен вариант `proctrack/cgroup`; для выбора ресурсов использован `select/cons_tres`; для планировщика выбран `sched/backfill`. Эти параметры соответствуют рекомендациям конфигуратора.

Один и тот же файл `slurm.conf` был размещён на всех узлах в каталоге `/etc/slurm`. На управляющем узле был создан каталог сохранения состояния контроллера, на вычислительных узлах был создан каталог для состояния `slurmd`. После запуска служб состояние кластера проверялось командами `sinfo` и `squeue`; перед запуском генерации вычислительные узлы должны находиться в состоянии `idle`.

Команда `sinfo` показывает состояние разделов SLURM и вычислительных узлов. Пример вывода для настроенного кластера:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
debug*	up	infinite	4	idle	cn[01-04]

Команда `squeue` используется для просмотра очереди заданий; перед запуском генератора её пустой вывод означает, что в очереди нет активных или ожидающих заданий.

⁴Slurm Configuration Tool на сайте SchedMD: <https://slurm.schedmd.com/configurator.html>.

2.4 Обработка исходного набора данных

Индивидуальный вариант определяет три параметра: дистрибутив openSUSE Leap, UID 50728 и имя задания qe7. Исходный набор данных был распакован и отфильтрован по UID и имени задания. В результате сформирован файл `acct_0921-0923_uid50728_qe7`, содержащий 1563 записи.

В выбранном фрагменте 1542 записи имеют состояние `COMPLETED`, 11 записей имеют состояние `FAILED`, 10 записей имеют состояние `CANCELLED`. Для построения распределений использовались завершённые задания, так как только для них поле длительности выполнения можно интерпретировать как корректное время работы задачи.

Исходная длительность выполнения задания берётся из поля `ElapsedRaw`. В выгрузке SLURM поле `TimelimitRaw` задано в минутах, а `ElapsedRaw` — в секундах, поэтому при вычислении ошибки прогноза запрошенное время переводится в секунды:

$$\Delta T = \text{TimelimitRaw} \cdot 60 - \text{ElapsedRaw}.$$

В выбранном наборе данных минимальная длительность завершённого задания составляет 699 секунд, медиана составляет 1652 секунды, среднее значение составляет 1712.39 секунды. Максимальное значение равно 16660 секунд. Ошибка прогноза времени для завершённых заданий имеет медиану 19948 секунд и среднее значение 19887.61 секунды; следовательно, пользовательские задания в данном варианте обычно запрашивали время с большим запасом относительно длительности из исходной статистики.

На рисунке 1 приведена сводная визуализация отфильтрованной выборки. Она показывает, сколько записей осталось после отбора по индивидуальному варианту, какую долю составляют успешно завершённые задания. Эти данные используются дальше при подборе распределений для генератора.

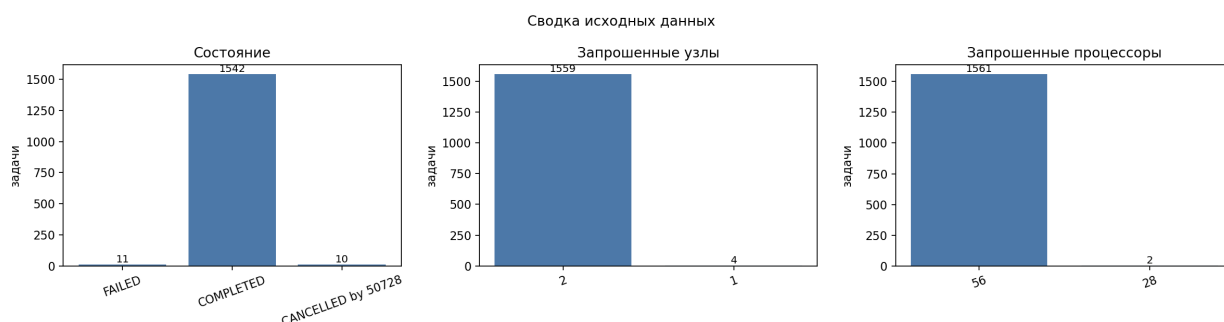


Рисунок 1. Сводка по исходному набору данных

2.5 Аппроксимация исходных распределений

Анализ исходных длительностей показал, что в данных присутствуют две выраженные группы заданий. Центр первой группы находится на уровне 1647 секунд, её вес равен 0.809; центр второй группы находится на уровне 1925 секунд, её вес равен 0.191. Поэтому длительность из исходного набора данных была описана смесью двух нормальных распределений. Параметры смеси приведены в таблице 1.

Таблица 1. Параметры смеси нормальных распределений исходной длительности

Компонента	μ , с	σ , с	Вес
1	1647.26	19.89	0.809
2	1924.82	18.70	0.191

Рисунок 2 показывает, что исходные длительности не образуют один общий пик: основная часть заданий сосредоточена вблизи 1650 секунд, меньшая группа — вблизи 1925 секунд. Наложенная кривая проходит через обе области концентрации, поэтому при генерации сохраняются два характерных режима выполнения, а не усреднённая длительность между ними.

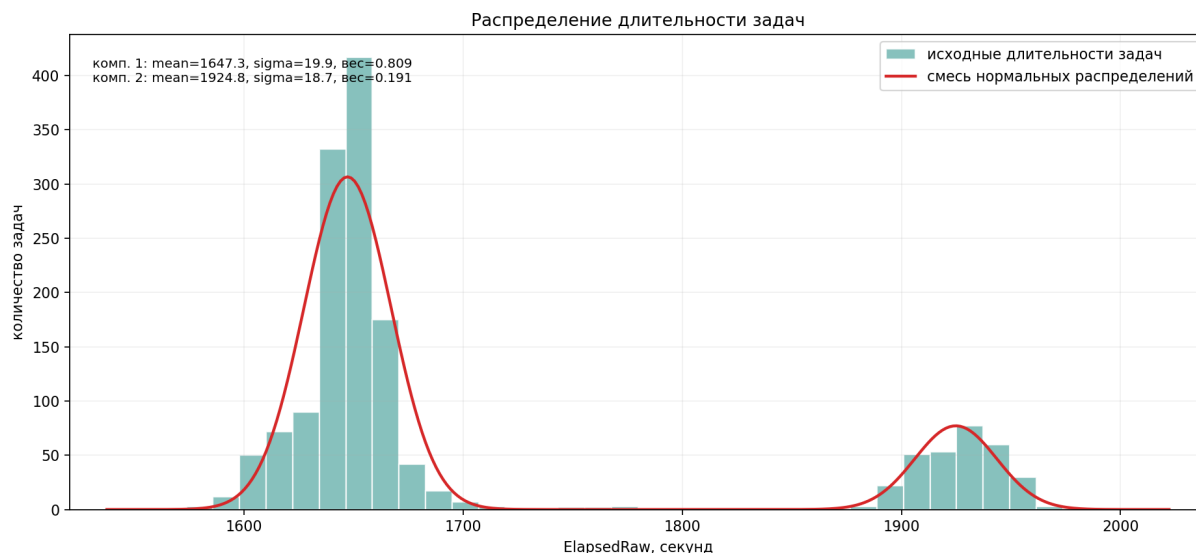


Рисунок 2. Аппроксимация распределения исходной длительности выполнения

В таблице 2 приведены параметры смеси логнормальных распределений для ошибки прогноза времени. Для каждой компоненты указаны параметры μ и σ в десятичной логарифмической шкале, а также вес компоненты в общей смеси. Первая компонента соответствует менее частой части выборки, вторая компонента является основной.

Таблица 2. Параметры смеси логнормальных распределений ошибки прогноза

Компонента	μ	σ	Вес
1	4.2939	0.0006	0.192
2	4.3000	0.0006	0.808

На рисунке 3 показан десятичный логарифм ошибки прогноза времени по исходному набору данных. Логарифмирование показывает, что основная масса значений разделяется на две близкие области; по ним заданы компоненты смеси из таблицы 2.

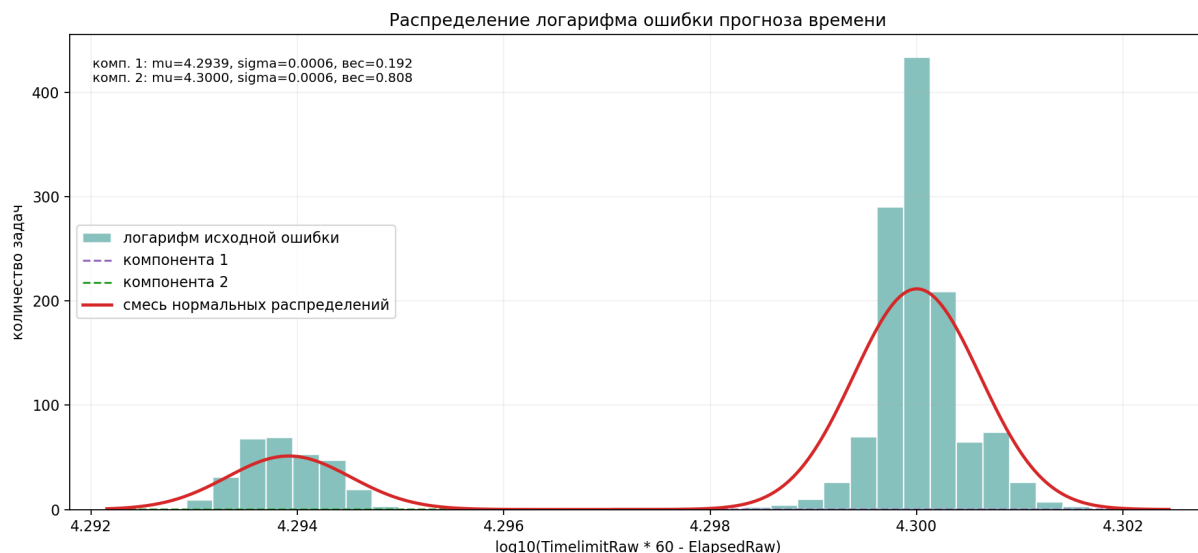


Рисунок 3. Распределение логарифма ошибки прогноза времени

На рисунке 4 показано распределение ошибки прогноза времени в масштабированной шкале запуска. Голубая гистограмма соответствует расчётной ошибке из манифеста.

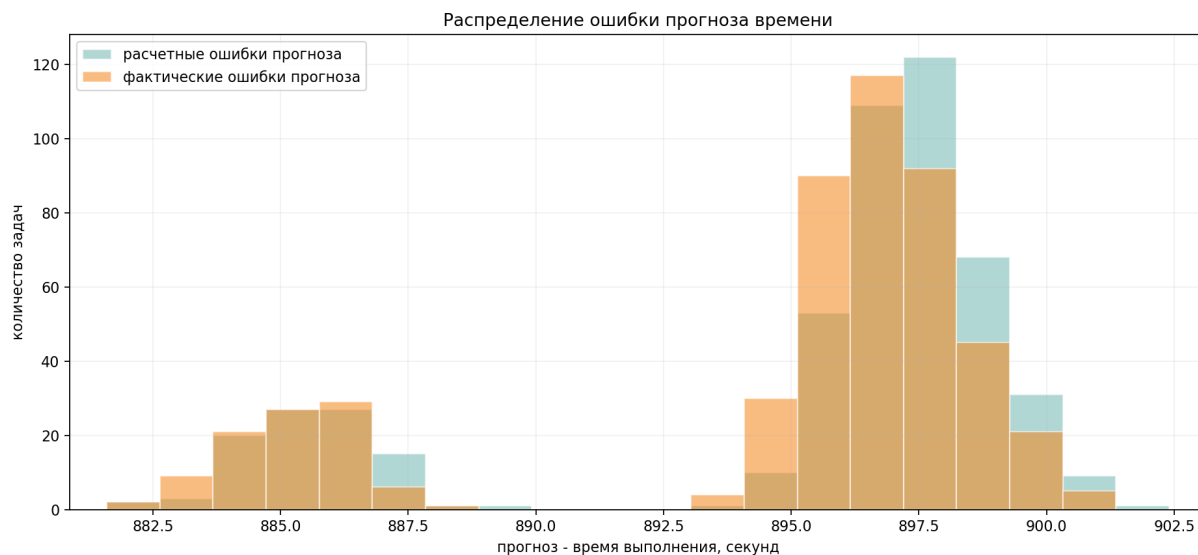


Рисунок 4. Распределение ошибки прогноза времени

На рисунке 4 также приведена фактическая ошибка после выполнения заданий в SLURM. Она показана оранжевой гистограммой и используется для сопоставления расчётной ошибки из манифеста с результатом запуска. Подробнее фактическая ошибка рассматривается в разделе 2.8.

2.6 Генерация заданий

Генератор запускается на управляющем узле `mgmt`. На вход ему передаются число заданий, масштаб длительности выполнения, масштаб планового времени и параметры распределений. Для каждого задания выбирается исходная длительность, ошибка прогноза и число требуемых узлов. Плановое время рассчитывается как сумма выбранной длительности и ошибки прогноза, после чего значения масштабируются для запуска на кластере.

Результатом работы генератора является каталог запуска с манифестом `manifest.csv` и набором файлов заданий `.slurm`. В манифест записываются имя файла задания, сгенерированные и масштабированные времена, лимит SLURM, число узлов и число задач. Каждый файл задания содержит параметры `#SBATCH`, команду `sleep` с рассчитанной длительностью и запись времени начала и окончания в общий журнал. Пример такого задания приведён в приложении А.

Масштабирование нужно для того, чтобы исходные задания длительностью в десятки минут можно было воспроизвести на кластере за ограниченное время. Масштаб длительности выполнения применяется к выбранной длительности задания и определяет аргумент команды `sleep`. Масштаб планового времени применяется к сумме выбранной длительности и ошибки прогноза; это значение записывается в манифест как плановое время и используется при расчёте ошибки прогноза. В параметре `--time` для SLURM передаётся то же плановое время, но округлённое до минут из-за формата лимита выполнения.

В манифесте для итогового запуска оба масштаба были выбраны 0.045. После масштабирования медиана времени `sleep` составила 74 секунды, среднее значение — 76.116 секунды, максимум — 89 секунд. Медиана планового времени составила 972 секунды. Так как длительность выполнения и плановое время масштабируются одинаково, соотношение между длительностью задания и запасом времени сохраняется.

В таблице 3 приведено распределение сгенерированных заданий по числу запрошенных узлов. Число узлов формируется по нормальному распределению и не привязано к значениям в исходной выборке. Большая часть заданий использует 2 узла, задания на 1 и 3 узла встречаются реже, задания на 4 узла составляют малую часть выборки.

Таблица 3. Распределение сгенерированных заданий по числу узлов

Число узлов	Число заданий
1	118
2	245
3	124
4	13

На рисунке 5 сравниваются исходное распределение длительностей и сгенерированная выборка. В сгенерированных данных сохраняются две основные группы заданий, при этом форма гистограммы отличается от исходной выборки из-за случайного выбора значений.

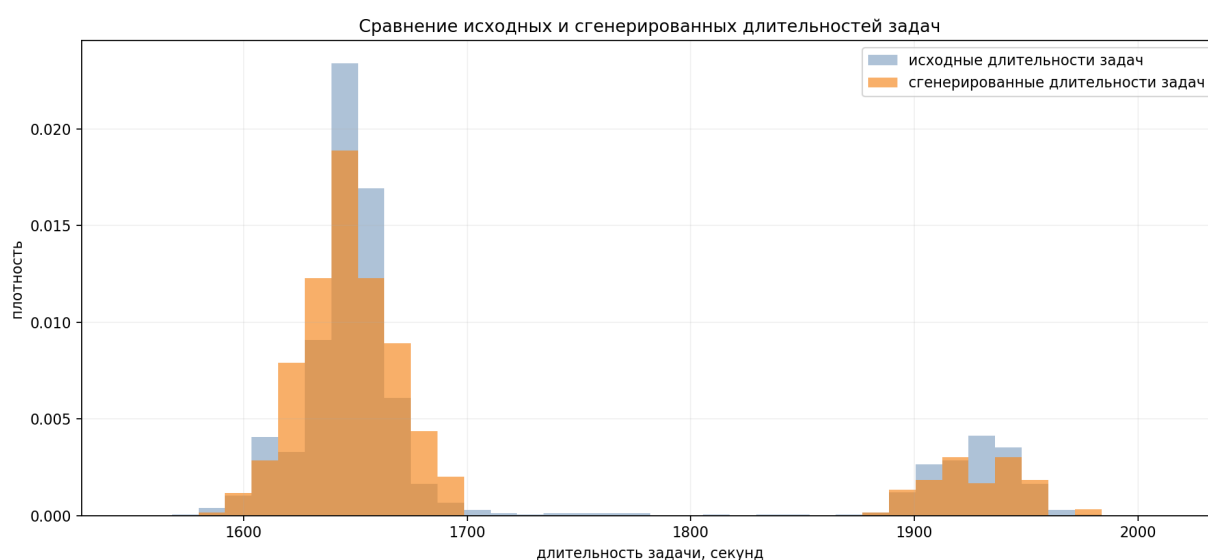


Рисунок 5. Сравнение исходной и сгенерированной длительности выполнения

2.7 Запуск заданий и сбор фактической статистики

После генерации задания отправляются в SLURM командой `sbatch`. Состояние очереди отслеживается через `squeue`, а сведения о завершённых заданиях сохраняются в файл `results.csv`. Для каждого задания фиксируются идентификатор SLURM, имя файла задания, состояние, узел выполнения, время ожидания в очереди, фактическое время выполнения и наблюдаемое полное время.

При анализе используются три временные величины. Запланированное время берётся из манифеста генератора. Значение `--time` передаётся в SLURM как лимит выполнения, но SLURM округляет его до минут. Фактическое время выполнения берётся из статистики SLURM. Поэтому фактическая ошибка прогноза считается по плановому значению из `manifest.csv`, а не по округлённому лимиту из очереди.

В итоговом запуске все 500 заданий завершились в состоянии COMPLETED. Запуск занял 6 часов 7 минут 4 секунды. Суммарное фактическое время выполнения всех заданий составило 10 часов 38 минут 28 секунд; оно больше времени всего запуска, так как задания выполнялись параллельно. Запланированное время `sleep` находилось в диапазоне от 72 до 89 секунд, медиана составила 74 секунды. Фактическое время выполнения находилось в диапазоне от 72 до 90 секунд, медиана составила 75 секунд. Медиана накладных расходов относительно команды `sleep` составила 0.562%, среднее значение — 0.656%, максимальное значение — 1.389%.

На рисунке 6 показано сравнение планового и фактического времени по отдельным заданиям.



Рисунок 6. Плановое и фактическое время выполнения заданий

Отклонение фактического времени по каждому заданию считается относительно времени `sleep`, записанного генератором в файл задания:

$$\delta_i = \frac{T_{\text{run},i}^{\text{SLURM}} - T_{\text{sleep},i}^{\text{manifest}}}{T_{\text{sleep},i}^{\text{manifest}}} \cdot 100\%.$$

Здесь $T_{\text{sleep},i}^{\text{manifest}}$ — запланированная длительность команды `sleep` для i -

го задания, а $T_{\text{run},i}^{\text{SLURM}}$ — фактическое время выполнения этого задания по статистике SLURM. На рисунке 7 показаны значения δ_i по всем заданиям. Они отражают накладные расходы запуска и завершения заданий в SLURM и виртуальной среде. В итоговом запуске все значения остаются ниже порога 3%.



Рисунок 7. Накладные расходы фактического выполнения по заданиям

Для сравнения был рассмотрен более короткий предварительный запуск на 200 заданий с масштабом времени 0.01, то есть с ускорением в 100 раз. В этом случае длительность `sleep` для большинства заданий попадает в диапазон 10–20 секунд, поэтому постоянные накладные расходы SLURM занимают заметную долю времени выполнения.



Рисунок 8. Накладные расходы при запуске 200 заданий с ускорением в 100 раз

На рисунке 8 часть значений выходит за контрольный порог 3%. В этом

запуске порог превысили 139 из 200 заданий, максимальное отклонение составило 12.5%, медианное значение — 5.0%. Поэтому для итогового запуска был выбран менее агрессивный масштаб времени: он увеличивает длительность отдельных заданий, но уменьшает относительный вклад накладных расходов.

2.8 Анализ фактической ошибки прогноза

Для удобства сопоставления на рисунке 9 повторно приведено распределение ошибки прогноза времени с наложением фактической ошибки после запуска заданий.

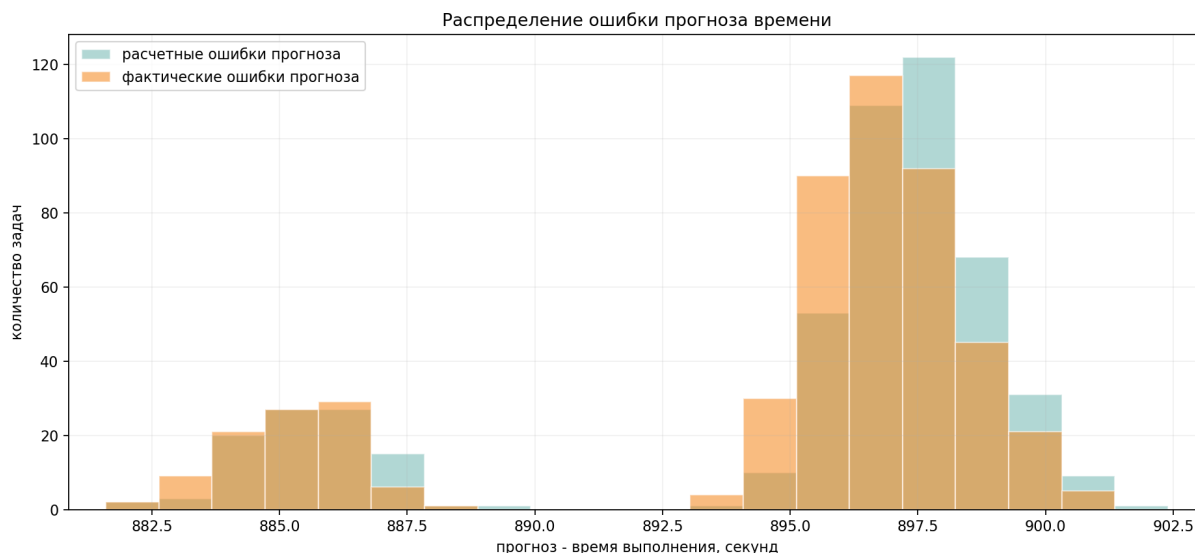


Рисунок 9. Сопоставление расчётной и фактической ошибки прогноза времени

Голубая гистограмма показывает запас времени, заложенный генератором до отправки заданий в очередь, оранжевая гистограмма показывает тот же запас после замены времени `sleep` на фактическое время выполнения из статистики SLURM. Распределения лежат в одном диапазоне, поэтому накладные расходы запуска не меняют характер ошибки прогноза, а только немного сдвигают отдельные значения.

После получения статистики SLURM ошибка прогноза пересчитывается для фактически выполненных заданий. Она определяется как разность между масштабированным плановым временем из манифеста и фактическим временем выполнения:

$$\Delta T_{\text{fact}} = T_{\text{plan}}^{\text{manifest}} - T_{\text{run}}^{\text{SLURM}}.$$

Здесь $T_{\text{plan}}^{\text{manifest}}$ — плановое время из манифеста генератора, а $T_{\text{run}}^{\text{SLURM}}$ — фактическое время выполнения, полученное из статистики SLURM.

На рисунке 10 показано распределение фактической ошибки прогноза. В запуске на 500 заданий ошибка находилась в диапазоне от 881 до 901 секунды, медиана составила 897 секунд. Наложенная кривая описывает наблюдаемую фактическую ошибку после выполнения заданий, а не параметры генератора.

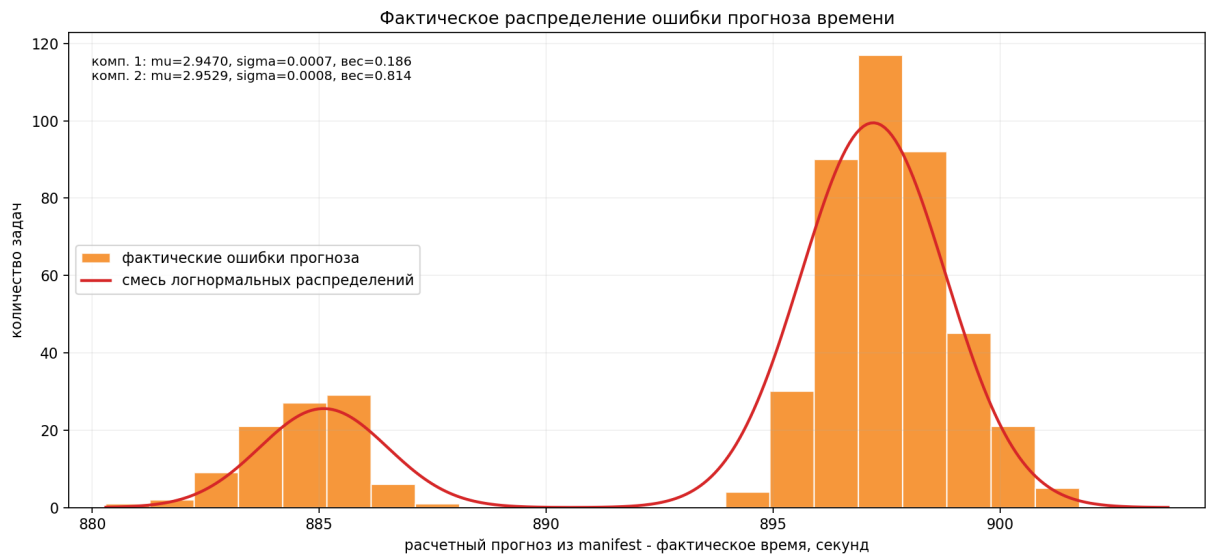


Рисунок 10. Распределение фактической ошибки прогноза времени

На рисунке 11 показан десятичный логарифм фактической ошибки прогноза. Фактические значения образуют компактную область рядом с распределением, заданным параметрами генератора. Небольшой сдвиг связан с тем, что фактическое время выполнения немного отличается от запланированного времени `sleep`.

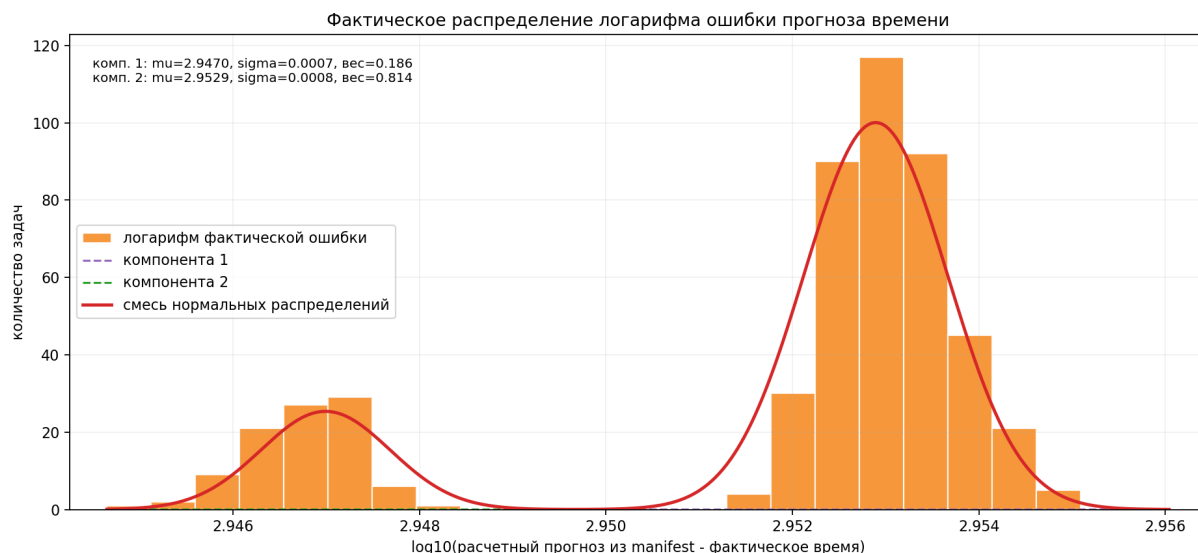


Рисунок 11. Распределение логарифма фактической ошибки прогноза времени

В таблице 4 сопоставлены параметры смеси, заданной генератором, и параметры смеси, рассчитанной по фактической ошибке после запуска. Исходные параметры приведены после масштабирования на коэффициент 0.045, поэтому значения μ находятся в той же десятичной логарифмической шкале, что и фактические данные. Центры компонент практически совпадают, а значение σ у фактической ошибки больше из-за накладных расходов запуска и округления времени выполнения в статистике SLURM.

Таблица 4. Сравнение параметров ошибки прогноза до и после запуска

Комп.	$\mu_{\text{исх.}}$	$\mu_{\text{факт.}}$	$\sigma_{\text{исх.}}$	$\sigma_{\text{факт.}}$	Вес исх.	Вес факт.
1	2.9481	2.9470	0.0006	0.0007	0.192	0.186
2	2.9532	2.9529	0.0006	0.0008	0.808	0.814

Итоговый запуск на 500 заданий показал, что выбранный масштаб времени подходит для проверки на четырёх вычислительных узлах. Медианное отклонение фактического времени от `sleep` составило 0.562%, среднее значение — 0.656%, максимальное значение — 1.389%. Фактическая ошибка прогноза остаётся близкой к расчётной ошибке из манифеста; это сопоставление показано на рисунке 9.

Заключение

В ходе работы был сгенерирован поток пользовательских заданий SLURM на основе статистики реальной очереди и подготовлена среда для проверки этого потока на вычислительном кластере. На локальной машине были созданы пять виртуальных машин с openSUSE Leap: управляющий узел `mgmt` и вычислительные узлы `cn01–cn04`. Для узлов настроены статические IP-адреса, разрешение имён, SSH-доступ по ключам и общая конфигурация MUNGE.

Запуск выполнялся на хостовой машине MacBook Air с процессором Apple M3, 8 вычислительными ядрами и 16 ГБ оперативной памяти под управлением macOS 26.5.1. Каждой виртуальной машине был выделен один виртуальный процессор, 2 ГБ оперативной памяти и виртуальный диск объёмом 16 ГБ.

По задачам, сформулированным в постановке, получены следующие результаты:

1. Создана схема из пяти виртуальных машин.
2. Настроены статическая адресация, разрешение имён и связность узлов.
3. Настроен SSH-доступ по ключам без ввода пароля.
4. Установлен MUNGE, подготовлен общий ключ и синхронизировано время.
5. Установлен и настроен SLURM.
6. Состояние вычислительных узлов проверено средствами SLURM.
7. Исходный набор данных отфильтрован по индивидуальному варианту.
8. Рассчитаны параметры распределений длительности выполнения и ошибки прогноза времени.
9. Реализован генератор пользовательских заявок SLURM.
10. Сгенерированные задания запущены на кластере, результаты выполнения собраны в журнал.
11. Исходные, сгенерированные и полученные при запуске характеристики сопоставлены в таблицах и на графиках.

Последний сформированный манифест содержит 500 заданий. При выбранном масштабе времени медианное плановое время составляет 972 секунды, медианное время `sleep` составляет 74 секунды. Итоговый запуск занял 6 часов 7 минут 4 секунды; медианное отклонение фактического времени от

`sleep` составило 0.562%, максимальное отклонение — 1.389%.

Таким образом, поставленная задача выполнена: кластер SLURM развёрнут, генератор пользовательских заявок реализован, исходные данные обработаны, а результаты генерации и выполнения представлены в виде численных характеристик и графиков.

Список использованных источников

- [1] Slurm Workload Manager Documentation [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/documentation.html> (дата обращения: 21.06.2026).
- [2] Slurm Quick Start Administrator Guide [Электронный ресурс] // SchedMD. URL: https://slurm.schedmd.com/quickstart_admin.html (дата обращения: 21.06.2026).
- [3] slurm.conf [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/slurm.conf.html> (дата обращения: 21.06.2026).
- [4] Slurm Configuration Tool [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/configurator.html> (дата обращения: 21.06.2026).
- [5] sbatch [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/sbatch.html> (дата обращения: 21.06.2026).
- [6] sinfo [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/sinfo.html> (дата обращения: 21.06.2026).
- [7] squeue [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/squeue.html> (дата обращения: 21.06.2026).
- [8] MUNGE Uid 'N' Gid Emporium [Электронный ресурс]. URL: <https://dun.github.io/munge/> (дата обращения: 21.06.2026).
- [9] openSUSE Leap Documentation [Электронный ресурс] // openSUSE. URL: <https://doc.opensuse.org/> (дата обращения: 18.06.2026).
- [10] Zypper usage [Электронный ресурс] // openSUSE. URL: https://en.opensuse.org/SDB:Zypper_usage (дата обращения: 18.06.2026).

- [11] `ssh_config(5)` [Электронный ресурс] // OpenBSD manual pages. URL: https://man.openbsd.org/ssh_config (дата обращения: 21.06.2026).
- [12] ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.

Приложение А. Пример сгенерированного задания SLURM

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=ge7_0001
3  #SBATCH --partition=debug
4  #SBATCH --nodes=2
5  #SBATCH --ntasks=2
6  #SBATCH --time=00:17:00
7  #SBATCH --output=generated/logs/%x-%j.out
8  #SBATCH --error=generated/logs/%x-%j.err
9
10 set -euo pipefail
11
12 mkdir -p "$(dirname "generated/logs/generated-jobs.log")"
13 start_time="$(date --iso-8601=seconds)"
14 echo "job_index=1 source_job_id=507280001 start=${start_time} planned_sleep=74" >>
15   ↪ "generated/logs/generated-jobs.log"
16 sleep 74
17 end_time="$(date --iso-8601=seconds)"
18 echo "job_index=1 source_job_id=507280001 end=${end_time} planned_sleep=74" >>
19   ↪ "generated/logs/generated-jobs.log"
```

Приложение Б. Код генератора пользовательских заявок

Файл `generate/main.py` содержит разбор параметров, выбор значений из распределений, масштабирование времени и запись манифеста.

```
1 from __future__ import annotations
2
3 import argparse
4 import csv
5 import math
6 import random
7 import shutil
8 from pathlib import Path
9 from generate.template import render_batch_script
10
11
12
13 def positive_int(value: str) -> int:
14     parsed = int(value)
15     if parsed <= 0:
16         raise argparse.ArgumentTypeError("value must be positive")
17     return parsed
18
19
20 def positive_float(value: str) -> float:
21     parsed = float(value)
22     if parsed <= 0:
23         raise argparse.ArgumentTypeError("value must be positive")
24     return parsed
25
26
27 def parse_component(value: str) -> tuple[float, float, float]:
28     parts = value.split(",")
29     if len(parts) != 3:
30         raise argparse.ArgumentTypeError("component must be center,sigma,weight")
31     center = float(parts[0])
32     sigma = float(parts[1])
33     weight = float(parts[2])
34     if sigma <= 0 or weight <= 0:
35         raise argparse.ArgumentTypeError("sigma and weight must be positive")
36     return center, sigma, weight
37
38
39 def choose_component(rng: random.Random, components: list[tuple[float, float, float]]) ->
40     ↪ tuple[float, float]:
41     total = sum(weight for _, _, weight in components)
42     roll = rng.random() * total
43     cumulative = 0.0
44     for center, sigma, weight in components:
45         cumulative += weight
46         if roll <= cumulative:
47             return center, sigma
48     center, sigma, _ = components[-1]
49     return center, sigma
50
51 def sample_lognormal_value(rng: random.Random, components: list[tuple[float, float, float]])
52     ↪ -> int:
53     mu, sigma = choose_component(rng, components)
54     return max(1, int(round(rng.lognormvariate(mu, sigma))))
55
56 def sample_normal_value(rng: random.Random, components: list[tuple[float, float, float]]) ->
57     ↪ int:
58     mean, sigma = choose_component(rng, components)
59     return max(1, int(round(rng.normalvariate(mean, sigma))))
60
61 def sample_node_count(rng: random.Random) -> int:
62     sampled = int(round(rng.normalvariate(2.0, 0.75)))
63     return min(4, max(1, sampled))
64
65
66 def format_slurm_time(total_seconds: int) -> str:
```

```

67     total_seconds = max(60, int(math.ceil(total_seconds / 60.0) * 60))
68     days, remainder = divmod(total_seconds, 24 * 60 * 60)
69     hours, remainder = divmod(remainder, 60 * 60)
70     minutes, seconds = divmod(remainder, 60)
71     if days:
72         return f"{days}-{hours:02d}:{minutes:02d}:{seconds:02d}"
73     return f"{hours:02d}:{minutes:02d}:{seconds:02d}"
74
75
76 def clean_run_dir(run_dir: Path) -> None:
77     if run_dir.exists():
78         shutil.rmtree(run_dir)
79     (run_dir / "slurm_jobs").mkdir(parents=True)
80
81
82 def generate_jobs(
83     run_dir: Path,
84     count: int,
85     sleep_scale: float,
86     time_scale: float,
87     seed: int,
88     partition: str,
89     runtime_components: list[tuple[float, float, float]],
90     error_components: list[tuple[float, float, float]],
91 ) -> Path:
92     clean_run_dir(run_dir)
93     rng = random.Random(seed)
94     slurm_jobs_dir = run_dir / "slurm_jobs"
95     manifest_path = run_dir / "manifest.csv"
96     log_file = Path("generated/logs/generated-jobs.log")
97
98     with manifest_path.open("w", newline="", encoding="utf-8") as manifest_file:
99         writer = csv.DictWriter(
100             manifest_file,
101             fieldnames=[
102                 "script",
103                 "source_job_id",
104                 "source_state",
105                 "source_elapsed_seconds",
106                 "sampled_elapsed_seconds",
107                 "sampled_error_seconds",
108                 "generated_forecast_seconds",
109                 "scaled_sleep_seconds",
110                 "scaled_forecast_seconds",
111                 "slurm_time",
112                 "nodes",
113                 "ntasks",
114             ],
115         )
116         writer.writeheader()
117
118         for index in range(1, count + 1):
119             sampled_elapsed_seconds = sample_normal_value(
120                 rng, runtime_components
121             )
122             sampled_error_seconds = sample_lognormal_value(rng, error_components)
123             generated_forecast_seconds = sampled_elapsed_seconds + sampled_error_seconds
124             scaled_sleep_seconds = max(1, int(round(sampled_elapsed_seconds * sleep_scale)))
125             scaled_forecast_seconds = max(
126                 60,
127                 int(round(generated_forecast_seconds * time_scale)),
128             )
129             nodes = sample_node_count(rng)
130             ntasks = nodes
131             slurm_time = format_slurm_time(scaled_forecast_seconds)
132             source_job_id = str(507280000 + index)
133             script_name = f"job_{index:04d}_{source_job_id}.slurm"
134             script_path = slurm_jobs_dir / script_name
135
136             script_path.write_text(
137                 render_batch_script(
138                     index=index,
139                     source_job_id=source_job_id,
140                     partition=partition,
141                     nodes=nodes,
142                     ntasks=ntasks,
143                     slurm_time=slurm_time,
144                     sleep_seconds=scaled_sleep_seconds,

```

```

145         log_file=log_file,
146     ),
147     encoding="utf-8",
148 )
149 script_path.chmod(0o755)
150
151 writer.writerow(
152     {
153         "script": script_name,
154         "source_job_id": source_job_id,
155         "source_state": "GENERATED",
156         "source_elapsed_seconds": sampled_elapsed_seconds,
157         "sampled_elapsed_seconds": sampled_elapsed_seconds,
158         "sampled_error_seconds": sampled_error_seconds,
159         "generated_forecast_seconds": generated_forecast_seconds,
160         "scaled_sleep_seconds": scaled_sleep_seconds,
161         "scaled_forecast_seconds": scaled_forecast_seconds,
162         "slurm_time": slurm_time,
163         "nodes": nodes,
164         "ntasks": ntasks,
165     }
166 )
167
168 return manifest_path
169
170
171 def parse_args() -> argparse.Namespace:
172     parser = argparse.ArgumentParser(description="Generate Slurm jobs on the mgmt VM.")
173     parser.add_argument("--output-root", type=Path, default=Path("generated/cluster_runs"))
174     parser.add_argument("--run-id", required=True)
175     parser.add_argument("--count", type=positive_int, default=200)
176     parser.add_argument("--sleep-scale", type=positive_float, default=0.01)
177     parser.add_argument("--time-scale", type=positive_float, default=0.01)
178     parser.add_argument("--seed", type=int, default=50728)
179     parser.add_argument("--partition", default="debug")
180     parser.add_argument(
181         "--runtime-normal-component",
182         action="append",
183         type=parse_component,
184         default=[],
185         help="Runtime normal component as mean,sigma,weight.",
186     )
187     parser.add_argument(
188         "--error-component",
189         action="append",
190         type=parse_component,
191         default=[],
192         help="Forecast error lognormal component as mu,sigma,weight.",
193     )
194     return parser.parse_args()
195
196
197 def main() -> None:
198     args = parse_args()
199     run_dir = (args.output_root / args.run_id).resolve()
200     runtime_components = args.runtime_normal_component or [(1701.49, 435.21, 1.0)]
201     error_components = args.error_component or [(9.8981, 0.0371, 1.0)]
202     generate_jobs(
203         run_dir=run_dir,
204         count=args.count,
205         sleep_scale=args.sleep_scale,
206         time_scale=args.time_scale,
207         seed=args.seed,
208         partition=args.partition,
209         runtime_components=runtime_components,
210         error_components=error_components,
211     )
212     print(run_dir)
213
214
215 if __name__ == "__main__":
216     main()

```

Файл generate/template.py формирует текст задания SLURM.

```
1 from __future__ import annotations
```

```

2
3 from pathlib import Path
4
5
6 def render_batch_script(
7     index: int,
8     source_job_id: str,
9     partition: str,
10    nodes: int,
11    ntasks: int,
12    slurm_time: str,
13    sleep_seconds: int,
14    log_file: Path,
15 ) -> str:
16     return f"""#!/usr/bin/env bash
17 #SBATCH --job-name=qe7_{index:04d}
18 #SBATCH --partition={partition}
19 #SBATCH --nodes={nodes}
20 #SBATCH --ntasks={ntasks}
21 #SBATCH --time={slurm_time}
22 #SBATCH --output=generated/logs/%x-%j.out
23 #SBATCH --error=generated/logs/%x-%j.err
24
25 set -euo pipefail
26
27 mkdir -p "$(dirname "{log_file}")"
28 start_time="$(date --iso-8601=seconds)"
29 echo "job_index={index} source_job_id={source_job_id} start=${{start_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
30 sleep {sleep_seconds}
31 end_time="$(date --iso-8601=seconds)"
32 echo "job_index={index} source_job_id={source_job_id} end=${{end_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
33 """

```

Приложение В. Конфигурация SLURM

```
1 # slurm.conf file generated by configurator.html.
2 # Put this file on all nodes of your cluster.
3 # See the slurm.conf man page for more information.
4 #
5 ClusterName=practice
6 SlurmctldHost=mgmt
7 #SlurmctldHost=
8 #
9 #DisableRootJobs=NO
10 #EnforcePartLimits=NO
11 #Epilog=
12 #EpilogSlurmctld=
13 #FirstJobId=1
14 #MaxJobId=67043328
15 #GresTypes=
16 #GroupUpdateForce=0
17 #GroupUpdateTime=600
18 #JobFileAppend=0
19 #JobRequeue=1
20 #JobSubmitPlugins=lua
21 #KillOnBadExit=0
22 #LaunchType=launch/slurm
23 #Licenses=foo*4,bar
24 MailProg=/bin/true
25 #MaxJobCount=10000
26 #MaxStepCount=40000
27 #MaxTasksPerNode=512
28 #MpiDefault=
29 #MpiParams=ports=-#-#
30 #PluginDir=
31 #PlugStackConfig=
32 #PrivateData=jobs
33 ProctrackType=proctrack/cgroup
34 #Prolog=
35 #PrologFlags=
36 #PrologSlurmctld=
37 #PropagatePrioProcess=0
38 #PropagateResourceLimits=
39 #PropagateResourceLimitsExcept=
40 #RebootProgram=
41 ReturnToService=2
42 SlurmctldPidFile=/var/run/slurmctld.pid
43 SlurmctldPort=6817
44 SlurmdPidFile=/var/run/slurmd.pid
45 SlurmdPort=6818
46 SlurmdSpoolDir=/var/spool/slurmd
47 SlurmUser=slurm
48 #SlurmdUser=root
49 #SrunEpilog=
50 #SrunProlog=
51 StateSaveLocation=/var/spool/slurmctld
52 #SwitchType=
53 #TaskEpilog=
54 TaskPlugin=task/affinity,task/cgroup
55 #TaskProlog=
56 #TopologyPlugin=topology/tree
57 #TmpFS=/tmp
58 #TrackWCKey=no
59 #TreeWidth=
60 #UnkillableStepProgram=
61 #UsePAM=0
62 #
63 #
64 # TIMERS
65 #BatchStartTimeout=10
66 #CompleteWait=0
67 #EpilogMsgTime=2000
68 #GetEnvTimeout=2
69 #HealthCheckInterval=0
70 #HealthCheckProgram=
71 #HealthCheckTimeout=60
72 InactiveLimit=0
73 KillWait=30
74 #MessageTimeout=10
75 #ResvOverRun=0
```

```

76 MinJobAge=300
77 #OverTimeLimit=0
78 SlurmctldTimeout=120
79 SlurmdTimeout=300
80 #UnkillableStepTimeout=60
81 #VSizeFactor=0
82 Waittime=0
83 #
84 #
85 # SCHEDULING
86 #DefMemPerCPU=0
87 #MaxMemPerCPU=0
88 #SchedulerTimeSlice=30
89 SchedulerType=sched/backfill
90 SelectType=select/cons_tres
91 #
92 #
93 # JOB PRIORITY
94 #PriorityFlags=
95 #PriorityType=priority/multifactor
96 #PriorityDecayHalfLife=
97 #PriorityCalcPeriod=
98 #PriorityFavorSmall=
99 #PriorityMaxAge=
100 #PriorityUsageResetPeriod=
101 #PriorityWeightAge=
102 #PriorityWeightFairshare=
103 #PriorityWeightJobSize=
104 #PriorityWeightPartition=
105 #PriorityWeightQOS=
106 #
107 #
108 # LOGGING AND ACCOUNTING
109 #AccountingStorageEnforce=0
110 #AccountingStorageHost=
111 #AccountingStoragePort=
112 #AccountingStorageType=
113 #AccountingStoreFlags=
114 #JobCompHost=
115 #JobCompLoc=
116 #JobCompParams=
117 #JobCompPass=
118 #JobCompPort=
119 JobCompType=jobcomp/none
120 #JobCompUser=
121 #JobContainerType=
122 JobAcctGatherFrequency=30
123 #JobAcctGatherType=
124 SlurmctldDebug=info
125 SlurmctldLogFile=/var/log/slurm/slurmctld.log
126 SlurmdDebug=info
127 SlurmdLogFile=/var/log/slurm/slurmd.log
128 #SlurmSchedLogFile=
129 #SlurmSchedLogLevel=
130 #DebugFlags=
131 #
132 #
133 # POWER SAVE SUPPORT FOR IDLE NODES (optional)
134 #SuspendProgram=
135 #ResumeProgram=
136 #SuspendTimeout=
137 #ResumeTimeout=
138 #ResumeRate=
139 #SuspendExcNodes=
140 #SuspendExcParts=
141 #SuspendRate=
142 #SuspendTime=
143 #
144 #
145 # COMPUTE NODES
146 NodeName=cn[01-04] NodeAddr=cn[01-04] CPUs=1 RealMemory=1900 State=UNKNOWN
147 PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP

```

Приложение Г. Код анализатора исходных и фактических данных, а также код точки запуска программы и анализатора

Файл `src/core/jobs.py`

```
1  from __future__ import annotations
2
3  import csv
4  import math
5  import os
6  import statistics
7  import warnings
8  from collections import Counter
9  from dataclasses import dataclass
10 from pathlib import Path
11
12 DEFAULT_DATASET = Path("dataset/acct_0921-0923_uid50728_qe7")
13 os.environ.setdefault("LOKY_MAX_CPU_COUNT", str(os.cpu_count() or 1))
14
15
16 @dataclass(frozen=True)
17 class SourceJob:
18     job_id: str
19     uid: str
20     job_name: str
21     partition: str
22     req_nodes: int
23     req_cpus: int
24     alloc_nodes: int
25     alloc_cpus: int
26     elapsed_seconds: int
27     timelimit_minutes: int
28     priority: int
29     state: str
30
31     @property
32     def forecast_seconds(self) -> int:
33         return self.timelimit_minutes * 60
34
35     @property
36     def forecast_error_seconds(self) -> int:
37         return self.forecast_seconds - self.elapsed_seconds
38
39
40 @dataclass(frozen=True)
41 class LognormalFit:
42     mu: float
43     sigma: float
44     count: int
45
46     def pdf(self, x: float) -> float:
47         if x <= 0:
48             return 0.0
49         denominator = x * self.sigma * math.sqrt(2.0 * math.pi)
50         exponent = -((math.log(x) - self.mu) ** 2) / (2.0 * self.sigma**2)
51         return math.exp(exponent) / denominator
52
53
54 @dataclass(frozen=True)
55 class LognormalMixture:
56     components: tuple[LognormalFit, ...]
57     weights: tuple[float, ...]
58
59     def pdf(self, x: float) -> float:
60         return sum(
61             weight * component.pdf(x)
62             for component, weight in zip(self.components, self.weights)
63         )
64
65
66 @dataclass(frozen=True)
67 class NormalFit:
```

```

68     mean: float
69     sigma: float
70     count: int
71
72     def pdf(self, x: float) -> float:
73         denominator = self.sigma * math.sqrt(2.0 * math.pi)
74         exponent = -((x - self.mean) ** 2) / (2.0 * self.sigma**2)
75         return math.exp(exponent) / denominator
76
77
78 @dataclass(frozen=True)
79 class NormalMixture:
80     components: tuple[NormalFit, ...]
81     weights: tuple[float, ...]
82
83     def pdf(self, x: float) -> float:
84         return sum(
85             weight * component.pdf(x)
86             for component, weight in zip(self.components, self.weights)
87         )
88
89
90 @dataclass(frozen=True)
91 class DatasetSummary:
92     jobs: int
93     state_counts: Counter[str]
94     req_nodes_counts: Counter[int]
95     req_cpus_counts: Counter[int]
96     elapsed_median: float
97     elapsed_mean: float
98     elapsed_min: int
99     elapsed_max: int
100
101
102 def read_jobs(path: Path = DEFAULT_DATASET) -> list[SourceJob]:
103     with path.open(newline="", encoding="utf-8") as input_file:
104         reader = csv.DictReader(input_file, delimiter="|")
105         jobs: list[SourceJob] = []
106         for row in reader:
107             if not row.get("JobIDRaw"):
108                 continue
109             jobs.append(
110                 SourceJob(
111                     job_id=row["JobIDRaw"],
112                     uid=row["UID"],
113                     job_name=row["JobName"],
114                     partition=row["Partition"],
115                     req_nodes=int(row["ReqNodes"]),
116                     req_cpus=int(row["ReqCPUS"]),
117                     alloc_nodes=int(row["AllocNodes"]),
118                     alloc_cpus=int(row["AllocCPUS"]),
119                     elapsed_seconds=int(row["ElapsedRaw"]),
120                     timelimit_minutes=int(row["TimelimitRaw"]),
121                     priority=int(row["Priority"]),
122                     state=row["State"],
123                 )
124             )
125     return jobs
126
127
128 def elapsed_times(jobs: list[SourceJob]) -> list[int]:
129     return [job.elapsed_seconds for job in jobs if job.elapsed_seconds > 0]
130
131
132 def completed_elapsed_times(jobs: list[SourceJob]) -> list[int]:
133     return [
134         job.elapsed_seconds
135         for job in jobs
136         if job.state == "COMPLETED" and job.elapsed_seconds > 0
137     ]
138
139
140 def forecast_errors(jobs: list[SourceJob]) -> list[int]:
141     return [job.forecast_error_seconds for job in jobs if job.forecast_error_seconds > 0]
142
143
144 def completed_forecast_errors(jobs: list[SourceJob]) -> list[int]:
145     return [

```

```

146         job.forecast_error_seconds
147     for job in jobs
148         if job.state == "COMPLETED" and job.forecast_error_seconds > 0
149 ]
150
151
152 def fit_lognormal(values: list[int] | list[float]) -> LognormalFit:
153     if len(values) < 2:
154         raise ValueError(
155             "need at least two positive values to fit a lognormal distribution"
156         )
157     log_values = [math.log(value) for value in values]
158     return LognormalFit(
159         mu=statistics.mean(log_values),
160         sigma=statistics.stdev(log_values),
161         count=len(values),
162     )
163
164
165 def trim_tails(
166     values: list[int] | list[float], lower_quantile: float = 0.01, upper_quantile: float =
167     ↪ 0.99
168 ) -> list[int] | list[float]:
169     if len(values) < 10:
170         return values
171     sorted_values = sorted(values)
172     lower_index = int(len(sorted_values) * lower_quantile)
173     upper_index = int(len(sorted_values) * upper_quantile)
174     trimmed = sorted_values[lower_index:upper_index]
175     return trimmed or values
176
177 def fit_lognormal_mixture(
178     values: list[int] | list[float],
179     component_count: int = 2,
180     trim_outliers: bool = True,
181 ) -> LognormalMixture:
182     fit_values = trim_tails(values) if trim_outliers else values
183     if len(fit_values) < component_count * 2:
184         fit = fit_lognormal(fit_values)
185         return LognormalMixture((fit,), (1.0,))
186
187     try:
188         os.environ.setdefault("LOKY_MAX_CPU_COUNT", str(os.cpu_count() or 1))
189         with warnings.catch_warnings():
190             warnings.filterwarnings(
191                 "ignore",
192                 category=UserWarning,
193                 module=r"joblib\.externals\.loky\.backend\.context",
194             )
195         warnings.filterwarnings(
196             "ignore", message="Could not find the number of physical cores.*"
197         )
198         import numpy as np
199         from sklearn.mixture import GaussianMixture
200     except ModuleNotFoundError as error:
201         raise SystemExit(
202             "scikit-learn is required for lognormal mixture fitting. "
203             "Install dependencies with: python3 -m pip install -r requirements.txt"
204         ) from error
205
206     log_values = np.array([[math.log(value)] for value in fit_values if value > 0])
207     model = GaussianMixture(
208         n_components=component_count,
209         covariance_type="full",
210         n_init=1,
211         random_state=50728,
212     )
213     with warnings.catch_warnings():
214         warnings.filterwarnings("ignore", message="Could not find the number of physical
215         ↪ cores.*")
216         model.fit(log_values)
217
218     components: list[LognormalFit] = []
219     weights: list[float] = []
220     for weight, mean, covariance in zip(
221         model.weights_, model.means_.ravel(), model.covariances_.reshape(-1)

```

```

222         components.append(
223             LognormalFit(mu=float(mean), sigma=math.sqrt(float(covariance)), count=0)
224         )
225         weights.append(float(weight))
226
227     ordered = sorted(zip(components, weights), key=lambda item: item[0].mu)
228     return LognormalMixture(
229         components=tuple(component for component, _ in ordered),
230         weights=tuple(weight for _, weight in ordered),
231     )
232
233
234 def fit_normal_mixture(
235     values: list[int] | list[float],
236     component_count: int = 2,
237     trim_outliers: bool = True,
238 ) -> NormalMixture:
239     fit_values = trim_tails(values) if trim_outliers else values
240     if len(fit_values) < component_count * 2:
241         return NormalMixture(
242             (
243                 NormalFit(
244                     mean=statistics.mean(fit_values),
245                     sigma=statistics.stdev(fit_values),
246                     count=len(fit_values),
247                 ),
248             ),
249             (1.0,),
250         )
251
252     try:
253         os.environ.setdefault("LOKY_MAX_CPU_COUNT", str(os.cpu_count() or 1))
254         with warnings.catch_warnings():
255             warnings.filterwarnings(
256                 "ignore",
257                 category=UserWarning,
258                 module=r"joblib\.externals\.loky\.backend\.context",
259             )
260             import numpy as np
261             from sklearn.mixture import GaussianMixture
262     except ModuleNotFoundError as error:
263         raise SystemExit(
264             "scikit-learn is required for normal mixture fitting. "
265             "Install dependencies with: python3 -m pip install -r requirements.txt"
266         ) from error
267
268     values_array = np.array([[value] for value in fit_values])
269     model = GaussianMixture(
270         n_components=component_count,
271         covariance_type="full",
272         n_init=1,
273         random_state=50728,
274     )
275     with warnings.catch_warnings():
276         warnings.filterwarnings("ignore")
277         model.fit(values_array)
278
279     components: list[NormalFit] = []
280     weights: list[float] = []
281     for weight, mean, covariance in zip(
282         model.weights_, model.means_.ravel(), model.covariances_.reshape(-1)
283     ):
284         components.append(
285             NormalFit(mean=float(mean), sigma=math.sqrt(float(covariance)), count=0)
286         )
287         weights.append(float(weight))
288
289     ordered = sorted(zip(components, weights), key=lambda item: item[0].mean)
290     return NormalMixture(
291         components=tuple(component for component, _ in ordered),
292         weights=tuple(weight for _, weight in ordered),
293     )
294
295
296 def fit_elapsed_lognormal(jobs: list[SourceJob]) -> LognormalFit:
297     return fit_lognormal(elapsed_times(jobs))
298
299

```

```

300 def fit_elapsed_normal_mixture(jobs: list[SourceJob]) -> NormalMixture:
301     return fit_normal_mixture(completed_elapsed_times(jobs), component_count=2)
302
303
304 def fit_elapsed_lognormal_mixture(jobs: list[SourceJob]) -> LognormalMixture:
305     return fit_lognormal_mixture(completed_elapsed_times(jobs), component_count=2)
306
307
308 def fit_forecast_error_lognormal(jobs: list[SourceJob]) -> LognormalFit:
309     return fit_lognormal(forecast_errors(jobs))
310
311
312 def fit_forecast_error_lognormal_mixture(jobs: list[SourceJob]) -> LognormalMixture:
313     return fit_lognormal_mixture(completed_forecast_errors(jobs), component_count=2)
314
315
316 def summarize_jobs(jobs: list[SourceJob]) -> DatasetSummary:
317     elapsed_values = elapsed_times(jobs)
318     return DatasetSummary(
319         jobs=len(jobs),
320         state_counts=Counter(job.state for job in jobs),
321         req_nodes_counts=Counter(job.req_nodes for job in jobs),
322         req_cpus_counts=Counter(job.req_cpus for job in jobs),
323         elapsed_median=statistics.median(elapsed_values),
324         elapsed_mean=statistics.mean(elapsed_values),
325         elapsed_min=min(elapsed_values),
326         elapsed_max=max(elapsed_values),
327     )

```

Файл src/analyze/analyze.py

```

1  from __future__ import annotations
2
3  from src.core.jobs import (
4      DEFAULT_DATASET,
5      SourceJob,
6      fit_elapsed_normal_mixture,
7      fit_forecast_error_lognormal_mixture,
8      read_jobs,
9      summarize_jobs,
10 )
11
12
13 def print_summary(jobs: list[SourceJob]) -> None:
14     summary = summarize_jobs(jobs)
15     elapsed_mixture = fit_elapsed_normal_mixture(jobs)
16     error_mixture = fit_forecast_error_lognormal_mixture(jobs)
17
18     print(f"jobs={summary.jobs}")
19     print("task_runtime_distribution=ElapsedRaw")
20     print("runtime_distribution=normal_mixture")
21     for index, (component, weight) in enumerate(
22         zip(elapsed_mixture.components, elapsed_mixture.weights), start=1
23     ):
24         print(f"runtime_component_{index}_mean={component.mean:.6f}")
25         print(f"runtime_component_{index}_sigma={component.sigma:.6f}")
26         print(f"runtime_component_{index}_weight={weight:.6f}")
27     print("forecast_error_distribution=TimelimitRaw * 60 - ElapsedRaw")
28     print("forecast_error_model=lognormal_mixture")
29     for index, (component, weight) in enumerate(
30         zip(error_mixture.components, error_mixture.weights), start=1
31     ):
32         print(f"error_component_{index}_mu={component.mu:.6f}")
33         print(f"error_component_{index}_sigma={component.sigma:.6f}")
34         print(f"error_component_{index}_weight={weight:.6f}")
35     print(f"median_elapsed_seconds={summary.elapsed_median:.0f}")
36     print(f"mean_elapsed_seconds={summary.elapsed_mean:.2f}")
37     print(f"min_elapsed_seconds={summary.elapsed_min}")
38     print(f"max_elapsed_seconds={summary.elapsed_max}")
39     print(
40         "states="
41         + ", ".join(f"{key}:{value}" for key, value in summary.state_counts.items())
42     )

```

Файл src/analyze/graphs.py

```

1  from __future__ import annotations
2
3  import csv
4  import datetime as dt
5  import math
6  import os
7  import re
8  import shutil
9  import statistics
10 from pathlib import Path
11
12 from src.core.jobs import (
13     LognormalFit,
14     LognormalMixture,
15     NormalMixture,
16     SourceJob,
17     elapsed_times,
18     fit_elapsed_normal_mixture,
19     fit_forecast_error_lognormal_mixture,
20     fit_lognormal,
21     fit_lognormal_mixture,
22     forecast_errors,
23     summarize_jobs,
24 )
25
26
27 def _import_pyplot():
28     if "MPLCONFIGDIR" not in os.environ:
29         cache_dir = Path("generated/matplotlib_cache")
30         cache_dir.mkdir(parents=True, exist_ok=True)
31         os.environ["MPLCONFIGDIR"] = str(cache_dir)
32     try:
33         import matplotlib.pyplot as plt
34     except ModuleNotFoundError as error:
35         raise SystemExit(
36             "matplotlib is required for graphs. Install dependencies with: "
37             "python3 -m pip install -r requirements.txt"
38         ) from error
39     return plt
40
41
42 def _histogram(
43     values: list[int] | list[float], bins: int
44 ) -> tuple[list[float], list[int], float]:
45     lower = min(values)
46     upper = max(values)
47     width = (upper - lower) / bins if upper != lower else 1.0
48     counts = [0] * bins
49     for value in values:
50         index = min(bins - 1, int((value - lower) / width))
51         counts[index] += 1
52     centers = [lower + width * (index + 0.5) for index in range(bins)]
53     return centers, counts, width
54
55
56 def _quantile(values: list[float] | list[int], q: float) -> float:
57     sorted_values = sorted(values)
58     if not sorted_values:
59         raise ValueError("cannot calculate quantile for an empty sequence")
60     index = (len(sorted_values) - 1) * q
61     lower = math.floor(index)
62     upper = math.ceil(index)
63     if lower == upper:
64         return float(sorted_values[lower])
65     return float(
66         sorted_values[lower] * (upper - index) + sorted_values[upper] * (index - lower)
67     )
68
69
70 def _main_x_limits(
71     values: list[int] | list[float],
72     lower_quantile: float = 0.02,
73     upper_quantile: float = 0.995,
74 ) -> tuple[float, float]:
75     lower_value = _quantile(values, lower_quantile)
76     upper_value = _quantile(values, upper_quantile)
77     padding = (upper_value - lower_value) * 0.15
78     lower = max(0.0, lower_value - padding)

```

```

79     upper = upper_value + padding
80     if upper <= lower:
81         return min(values), max(values)
82     return lower, upper
83
84
85 def _padded_x_limits(
86     x_limits: tuple[float, float], padding_ratio: float = 0.035
87 ) -> tuple[float, float]:
88     lower, upper = x_limits
89     padding = (upper - lower) * padding_ratio
90     return lower - padding, upper + padding
91
92
93 def _compact_hist_bins(
94     values: list[int] | list[float],
95     max_bins: int,
96     target_bin_width: float = 1.2,
97     min_bins: int = 24,
98 ) -> int:
99     lower = min(values)
100    upper = max(values)
101    if upper <= lower:
102        return 1
103    compact_bins = math.ceil((upper - lower) / target_bin_width)
104    return min(max_bins, max(min_bins, compact_bins))
105
106
107 def _plot_pdf(
108     ax,
109     fit: LognormalFit | LognormalMixture,
110     values: list[int] | list[float],
111     bins: int,
112     label: str = "логнормальная аппроксимация",
113     x_limits: tuple[float, float] | None = None,
114 ) -> None:
115     if x_limits:
116         lower, upper = x_limits
117     else:
118         lower = max(1, min(values))
119         upper = max(values)
120     lower = max(1, lower)
121     step = (upper - lower) / 300
122     xs = [lower + step * index for index in range(301)]
123     bin_width = (upper - lower) / bins if upper != lower else 1.0
124     ys = [fit.pdf(x) * len(values) * bin_width for x in xs]
125     ax.plot(xs, ys, color="#d62728", linewidth=2.0, label=label)
126
127
128 def _plot_normal_mixture_on_values(
129     ax,
130     mixture: NormalMixture,
131     values: list[int] | list[float],
132     bins: int,
133     label: str = "смесь нормальных распределений",
134     x_limits: tuple[float, float] | None = None,
135 ) -> None:
136     if x_limits:
137         lower, upper = x_limits
138     else:
139         lower = min(values)
140         upper = max(values)
141     step = (upper - lower) / 300 if upper != lower else 1.0
142     xs = [lower + step * index for index in range(301)]
143     bin_width = (upper - lower) / bins if upper != lower else 1.0
144     ys = [mixture.pdf(x) * len(values) * bin_width for x in xs]
145     ax.plot(xs, ys, color="#d62728", linewidth=2.0, label=label)
146
147
148 def _normal_pdf(x: float, mu: float, sigma: float) -> float:
149     denominator = sigma * math.sqrt(2.0 * math.pi)
150     exponent = -((x - mu) ** 2) / (2.0 * sigma**2)
151     return math.exp(exponent) / denominator
152
153
154 def _plot_normal_pdf(
155     ax,
156     values: list[float],

```

```

157     mu: float,
158     sigma: float,
159     bins: int,
160     label: str = "нормальная аппроксимация",
161     x_limits: tuple[float, float] | None = None,
162 ) -> None:
163     if x_limits:
164         lower, upper = x_limits
165     else:
166         lower = min(values)
167         upper = max(values)
168     step = (upper - lower) / 300 if upper != lower else 1.0
169     xs = [lower + step * index for index in range(301)]
170     bin_width = (upper - lower) / bins if upper != lower else 1.0
171     ys = [_normal_pdf(x, mu, sigma) * len(values) * bin_width for x in xs]
172     ax.plot(xs, ys, color="#d62728", linewidth=2.0, label=label)
173
174
175 def _plot_normal_mixture_pdf(
176     ax,
177     mixture: LognormalMixture,
178     values: list[float],
179     bins: int,
180     x_limits: tuple[float, float] | None = None,
181 ) -> None:
182     if x_limits:
183         lower, upper = x_limits
184     else:
185         lower = min(values)
186         upper = max(values)
187     step = (upper - lower) / 300 if upper != lower else 1.0
188     xs = [lower + step * index for index in range(301)]
189     bin_width = (upper - lower) / bins if upper != lower else 1.0
190
191     total_ys = [0.0 for _ in xs]
192     colors = ["#9467bd", "#2ca02c"]
193     for index, (component, weight) in enumerate(
194         zip(mixture.components, mixture.weights), start=1
195     ):
196         ys = [
197             weight
198             * _normal_pdf(x, component.mu, component.sigma)
199             * len(values)
200             * bin_width
201             for x in xs
202         ]
203         total_ys = [total + value for total, value in zip(total_ys, ys)]
204     ax.plot(
205         xs,
206         total_ys,
207         color=colors[(index - 1) % len(colors)],
208         linewidth=1.4,
209         linestyle="--",
210         label=f"компонента {index}",
211     )
212
213     ax.plot(
214         xs,
215         total_ys,
216         color="#d62728",
217         linewidth=2.2,
218         label="смесь нормальных распределений",
219     )
220
221
222 def _log_values(values: list[float] | list[int], base: float) -> list[float]:
223     if base <= 0 or base == 1:
224         raise ValueError("log base must be positive and not equal to 1")
225     return [math.log(value, base) for value in values]
226
227
228 def _log_axis_label(base: float, expression: str) -> str:
229     if math.isclose(base, math.e):
230         return f"ln({expression})"
231     if float(base).is_integer():
232         return f"log{int(base)}({expression})"
233     return f"log base {base:g}({expression})"
234

```

```

235
236 def _rescale_lognormal_mixture(
237     mixture: LognormalMixture, base: float
238 ) -> LognormalMixture:
239     if math.isclose(base, math.e):
240         return mixture
241     scale = math.log(base)
242     return LognormalMixture(
243         components=tuple(
244             LognormalFit(
245                 mu=component.mu / scale,
246                 sigma=component.sigma / scale,
247                 count=component.count,
248             )
249             for component in mixture.components
250         ),
251         weights=mixture.weights,
252     )
253
254
255 def _mixture_text(mixture: LognormalMixture) -> str:
256     return "\n".join(
257         f"комп. {index}: mu={component.mu:.4f}, sigma={component.sigma:.4f}, wec={weight:.3f}"
258         for index, (component, weight) in enumerate(
259             zip(mixture.components, mixture.weights), start=1
260         )
261     )
262
263
264 def _normal_mixture_text(mixture: NormalMixture) -> str:
265     return "\n".join(
266         f"комп. {index}: mean={component.mean:.1f}, sigma={component.sigma:.1f},
267         ↪ wec={weight:.3f}"
268         for index, (component, weight) in enumerate(
269             zip(mixture.components, mixture.weights), start=1
270         )
271     )
272
273 def _style_large_plot(fig, ax) -> None:
274     ax.grid(True, alpha=0.18)
275     ax.margins(x=0.01)
276     fig.subplots_adjust(left=0.075, right=0.985, top=0.9, bottom=0.14)
277
278
279 def plot_source_summary(jobs: list[SourceJob], output_dir: Path) -> Path:
280     plt = _import_pyplot()
281     output_dir.mkdir(parents=True, exist_ok=True)
282     summary = summarize_jobs(jobs)
283     path = output_dir / "source_summary.png"
284
285     fig, axes = plt.subplots(1, 3, figsize=(15, 4))
286     charts = [
287         ("Состояние", summary.state_counts),
288         ("Запрошенные узлы", summary.req_nodes_counts),
289         ("Запрошенные процессоры", summary.req_cpus_counts),
290     ]
291     for ax, (title, counts) in zip(axes, charts):
292         labels = [str(key) for key in counts.keys()]
293         values = list(counts.values())
294         ax.bar(labels, values, color="#4c78a8")
295         ax.set_title(title)
296         ax.set_ylabel("задачи")
297         ax.tick_params(axis="x", rotation=20)
298         for index, value in enumerate(values):
299             ax.text(index, value, str(value), ha="center", va="bottom", fontsize=9)
300
301     fig.suptitle("Сводка исходных данных")
302     fig.tight_layout()
303     fig.savefig(path, dpi=160)
304     plt.close(fig)
305     return path
306
307
308 def clean_graphs_dir(output_dir: Path) -> None:
309     output_dir.mkdir(parents=True, exist_ok=True)
310     for path in output_dir.iterdir():
311         if path.is_dir():

```

```

312         shutil.rmtree(path)
313     else:
314         path.unlink()
315
316
317 def plot_runtime_fit(jobs: list[SourceJob], output_dir: Path, bins: int = 40) -> Path:
318     plt = _import_pyplot()
319     output_dir.mkdir(parents=True, exist_ok=True)
320     path = output_dir / "runtime_fit.png"
321     runtimes = elapsed_times(jobs)
322     mixture = fit_elapsed_normal_mixture(jobs)
323     x_limits = _main_x_limits(runtimes)
324
325     fig, ax = plt.subplots(figsize=(12, 6))
326     ax.hist(
327         runtimes,
328         bins=bins,
329         range=x_limits,
330         color="#72b7b2",
331         edgecolor="white",
332         alpha=0.85,
333         label="исходные длительности задач",
334     )
335     _plot_normal_mixture_on_values(
336         ax,
337         mixture,
338         runtimes,
339         bins,
340         label="смесь нормальных распределений",
341         x_limits=x_limits,
342     )
343     ax.set_title("Распределение длительности задач")
344     ax.set_xlabel("ElapsedRaw, секунд")
345     ax.set_ylabel("количество задач")
346     ax.set_xlim(*_padded_x_limits(x_limits))
347     ax.legend()
348
349     text = _normal_mixture_text(mixture)
350     ax.text(0.02, 0.95, text, transform=ax.transAxes, va="top", fontsize=9)
351     _style_large_plot(fig, ax)
352     fig.savefig(path, dpi=160)
353     plt.close(fig)
354     return path
355
356
357 def _manifest_scaled_forecast_errors(manifest_path: Path | None) -> list[float]:
358     if not manifest_path or not manifest_path.exists():
359         return []
360     errors: list[float] = []
361     with manifest_path.open(newline="", encoding="utf-8") as input_file:
362         for row in csv.DictReader(input_file):
363             if row.get("scaled_forecast_seconds"):
364                 forecast = float(row["scaled_forecast_seconds"])
365             else:
366                 sampled_elapsed = float(row["sampled_elapsed_seconds"])
367                 if sampled_elapsed <= 0:
368                     continue
369                 scale = float(row["scaled_sleep_seconds"]) / sampled_elapsed
370                 forecast = float(row["generated_forecast_seconds"]) * scale
371             error = forecast - float(row["scaled_sleep_seconds"])
372             if error > 0:
373                 errors.append(error)
374     return errors
375
376
377 def plot_forecast_error_fit(
378     jobs: list[SourceJob],
379     output_dir: Path,
380     manifest_path: Path | None = None,
381     cluster_results_path: Path | None = None,
382     bins: int = 40,
383 ) -> Path:
384     plt = _import_pyplot()
385     output_dir.mkdir(parents=True, exist_ok=True)
386     path = output_dir / "forecast_error_fit.png"
387     errors = _manifest_scaled_forecast_errors(manifest_path) or forecast_errors(jobs)
388     actual_errors: list[float] = []
389     if cluster_results_path:

```

```

390     actual_errors = _cluster_actual_errors_from_manifest(
391         _read_cluster_runtime_rows(cluster_results_path), manifest_path
392     )
393
394     x_values = errors + actual_errors if actual_errors else errors
395     x_limits = _main_x_limits(x_values, upper_quantile=0.98)
396     histogram_bins = _compact_hist_bins(
397         x_values, bins, target_bin_width=1.2, min_bins=20
398     )
399
400     fig, ax = plt.subplots(figsize=(12, 6))
401     ax.hist(
402         errors,
403         bins=histogram_bins,
404         range=x_limits,
405         color="#72b7b2",
406         edgecolor="white",
407         alpha=0.55,
408         label="расчетные ошибки прогноза",
409     )
410     if actual_errors:
411         ax.hist(
412             actual_errors,
413             bins=histogram_bins,
414             range=x_limits,
415             color="#f58518",
416             edgecolor="white",
417             alpha=0.55,
418             label="фактические ошибки прогноза",
419         )
420     ax.set_title("Распределение ошибки прогноза времени")
421     ax.set_xlabel("прогноз - время выполнения, секунд")
422     ax.set_ylabel("количество задач")
423     ax.set_xlim(*_padded_x_limits(x_limits))
424     ax.legend()
425
426     _style_large_plot(fig, ax)
427     fig.savefig(path, dpi=160)
428     plt.close(fig)
429     return path
430
431
432 def plot_log_forecast_error_fit(
433     jobs: list[SourceJob], output_dir: Path, bins: int = 40, log_base: float = math.e
434 ) -> Path:
435     plt = _import_pyplot()
436     output_dir.mkdir(parents=True, exist_ok=True)
437     path = output_dir / "log_forecast_error_fit.png"
438     errors = forecast_errors(jobs)
439     log_errors = _log_values(errors, log_base)
440     mixture = _rescale_lognormal_mixture(
441         fit_forecast_error_lognormal_mixture(jobs), log_base
442     )
443     x_limits = _main_x_limits(log_errors, upper_quantile=0.98)
444
445     fig, ax = plt.subplots(figsize=(12, 6))
446     ax.hist(
447         log_errors,
448         bins=bins,
449         range=x_limits,
450         color="#72b7b2",
451         edgecolor="white",
452         alpha=0.85,
453         label="логарифм исходной ошибки",
454     )
455     _plot_normal_mixture_pdf(ax, mixture, log_errors, bins, x_limits=x_limits)
456     ax.set_title("Распределение логарифма ошибки прогноза времени")
457     ax.set_xlabel(_log_axis_label(log_base, "TimelimitRaw * 60 - ElapsedRaw"))
458     ax.set_ylabel("количество задач")
459     ax.set_xlim(*_padded_x_limits(x_limits))
460     ax.legend()
461
462     text = _mixture_text(mixture)
463     ax.text(0.02, 0.95, text, transform=ax.transAxes, va="top", fontsize=9)
464     _style_large_plot(fig, ax)
465     fig.savefig(path, dpi=160)
466     plt.close(fig)
467     return path

```

```

468
469
470 def plot_generated_vs_source(
471     jobs: list[SourceJob], manifest_path: Path, output_dir: Path, bins: int = 40
472 ) -> Path:
473     plt = _import_pyplot()
474     output_dir.mkdir(parents=True, exist_ok=True)
475     path = output_dir / "generated_runtime_vs_source.png"
476
477     source_runtimes = elapsed_times(jobs)
478     generated_runtimes = []
479     with manifest_path.open(newline="", encoding="utf-8") as input_file:
480         for row in csv.DictReader(input_file):
481             generated_runtimes.append(int(row["sampled_elapsed_seconds"]))
482     x_limits = _main_x_limits(source_runtimes + generated_runtimes)
483
484     fig, ax = plt.subplots(figsize=(12, 6))
485     ax.hist(
486         source_runtimes,
487         bins=bins,
488         range=x_limits,
489         density=True,
490         color="#4c78a8",
491         alpha=0.45,
492         label="исходные длительности задач",
493     )
494     ax.hist(
495         generated_runtimes,
496         bins=bins,
497         range=x_limits,
498         density=True,
499         color="#f58518",
500         alpha=0.65,
501         label="сгенерированные длительности задач",
502     )
503     ax.set_title("Сравнение исходных и сгенерированных длительностей задач")
504     ax.set_xlabel("длительность задачи, секунд")
505     ax.set_ylabel("плотность")
506     ax.set_xlim(*_padded_x_limits(x_limits))
507     ax.legend()
508     _style_large_plot(fig, ax)
509     fig.savefig(path, dpi=160)
510     plt.close(fig)
511     return path
512
513
514 def _parse_log_times(log_path: Path) -> dict[str, dict[str, dt.datetime]]:
515     pattern = re.compile(
516         r"job_index=(?P<index>\d+) .* (?P<kind>start|end)=(?P<time>\\S+)"
517     )
518     result: dict[str, dict[str, dt.datetime]] = {}
519     if not log_path.exists():
520         return result
521     for line in log_path.read_text(encoding="utf-8").splitlines():
522         match = pattern.search(line)
523         if not match:
524             continue
525         index = match.group("index")
526         result.setdefault(index, {}) [match.group("kind")] = dt.datetime.fromisoformat(
527             match.group("time")
528         )
529     return result
530
531
532 def plot_execution_overhead(
533     manifest_path: Path, log_path: Path, output_dir: Path
534 ) -> Path | None:
535     plt = _import_pyplot()
536     log_times = _parse_log_times(log_path)
537     if not log_times:
538         return None
539
540     planned = {}
541     with manifest_path.open(newline="", encoding="utf-8") as input_file:
542         for row in csv.DictReader(input_file):
543             index = row["script"].split("_")[1]
544             planned[index] = int(row["scaled_sleep_seconds"])
545

```

```

546 indexes: list[int] = []
547 overhead_percent: list[float] = []
548 for index, times in sorted(log_times.items(), key=lambda item: int(item[0])):
549     if "start" not in times or "end" not in times or index not in planned:
550         continue
551     actual_seconds = (times["end"] - times["start"]).total_seconds()
552     expected_seconds = planned[index]
553     if expected_seconds <= 0:
554         continue
555     indexes.append(int(index))
556     overhead_percent.append(
557         (actual_seconds - expected_seconds) / expected_seconds * 100.0
558     )
559
560 if not indexes:
561     return None
562
563 output_dir.mkdir(parents=True, exist_ok=True)
564 path = output_dir / "execution_overhead.png"
565 fig, ax = plt.subplots(figsize=(10, 5))
566 ax.bar(indexes, overhead_percent, color="#54a24b")
567 ax.axhline(3.0, color="#d62728", linestyle="--", linewidth=1.5, label="порог 3%")
568 ax.set_title("Накладные расходы фактического запуска")
569 ax.set_xlabel("номер сгенерированной задачи")
570 ax.set_ylabel("(факт - план) / план, %")
571 ax.legend()
572 fig.tight_layout()
573 fig.savefig(path, dpi=160)
574 plt.close(fig)
575 return path
576
577
578 def _read_cluster_runtime_rows(results_path: Path) -> list[dict[str, str]]:
579     if not results_path.exists():
580         return []
581
582     rows = []
583     with results_path.open(newline="", encoding="utf-8") as input_file:
584         for row in csv.DictReader(input_file):
585             if not row.get("actual_slurm_runtime_seconds"):
586                 continue
587             rows.append(row)
588     return rows
589
590
591 def _parse_slurm_time(value: str) -> int:
592     if "-" in value:
593         days_raw, time_raw = value.split("-", 1)
594         days = int(days_raw)
595     else:
596         days = 0
597         time_raw = value
598     hours_raw, minutes_raw, seconds_raw = time_raw.split(":")
599     return (
600         days * 24 * 60 * 60
601         + int(hours_raw) * 60 * 60
602         + int(minutes_raw) * 60
603         + int(seconds_raw)
604     )
605
606
607 def _cluster_runtime_series(
608     rows: list[dict[str, str]],
609 ) -> tuple[list[int], list[float], list[float]]:
610     sorted_rows = sorted(rows, key=lambda row: int(row["script"].split("_")[1]))
611     indexes = [int(row["script"].split("_")[1]) for row in sorted_rows]
612     planned = [float(row["planned_sleep_seconds"]) for row in sorted_rows]
613     actual = [float(row["actual_slurm_runtime_seconds"]) for row in sorted_rows]
614     overhead = [float(row["overhead_vs_sleep_percent"]) for row in sorted_rows]
615     return indexes, planned, actual, overhead
616
617
618 def _cluster_actual_errors(rows: list[dict[str, str]]) -> list[float]:
619     errors: list[float] = []
620     for row in rows:
621         if not row.get("requested_slurm_time") or not row.get(
622             "actual_slurm_runtime_seconds"
623         ):

```

```

624         continue
625         error = _parse_slurm_time(row["requested_slurm_time"]) - float(
626             row["actual_slurm_runtime_seconds"]
627         )
628         if error > 0:
629             errors.append(error)
630     return errors
631
632
633 def _manifest_scaled_forecasts(manifest_path: Path | None) -> dict[str, float]:
634     if not manifest_path or not manifest_path.exists():
635         return {}
636     result: dict[str, float] = {}
637     with manifest_path.open(newline="", encoding="utf-8") as input_file:
638         for row in csv.DictReader(input_file):
639             if row.get("scaled_forecast_seconds"):
640                 result[row["script"]] = float(row["scaled_forecast_seconds"])
641                 continue
642                 sampled_elapsed = float(row["sampled_elapsed_seconds"])
643                 if sampled_elapsed <= 0:
644                     continue
645                 scale = float(row["scaled_sleep_seconds"]) / sampled_elapsed
646                 result[row["script"]] = float(row["generated_forecast_seconds"]) * scale
647     return result
648
649
650 def _cluster_actual_errors_from_manifest(
651     rows: list[dict[str, str]], manifest_path: Path | None
652 ) -> list[float]:
653     forecasts = _manifest_scaled_forecasts(manifest_path)
654     errors: list[float] = []
655     for row in rows:
656         forecast = forecasts.get(row["script"])
657         if forecast is None or not row.get("actual_slurm_runtime_seconds"):
658             continue
659         error = forecast - float(row["actual_slurm_runtime_seconds"])
660         if error > 0:
661             errors.append(error)
662     return errors
663
664
665 def plot_actual_error_fit(
666     results_path: Path,
667     output_dir: Path,
668     manifest_path: Path | None = None,
669     bins: int = 30,
670     log_base: float = math.e,
671 ) -> Path | None:
672     plt = _import_pyplot()
673     rows = _read_cluster_runtime_rows(results_path)
674     errors = _cluster_actual_errors_from_manifest(rows, manifest_path)
675
676     if len(errors) < 2:
677         return None
678
679     mixture = fit_lognormal_mixture(errors, component_count=2)
680     log_mixture = _rescale_lognormal_mixture(mixture, log_base)
681
682     output_dir.mkdir(parents=True, exist_ok=True)
683     path = output_dir / "actual_error_fit.png"
684     x_limits = _main_x_limits(errors)
685     histogram_bins = _compact_hist_bins(errors, bins)
686
687     fig, ax = plt.subplots(figsize=(12, 6))
688     ax.hist(
689         errors,
690         bins=histogram_bins,
691         range=x_limits,
692         color="#f58518",
693         edgecolor="white",
694         alpha=0.85,
695         label="фактические ошибки прогноза",
696     )
697     _plot_pdf(
698         ax,
699         mixture,
700         errors,
701         histogram_bins,

```

```

702         label="смесь логнормальных распределений",
703         x_limits=x_limits,
704     )
705     ax.set_title("Фактическое распределение ошибки прогноза времени")
706     ax.set_xlabel("расчетный прогноз из manifest - фактическое время, секунд")
707     ax.set_ylabel("количество задач")
708     ax.set_xlim(*_padded_x_limits(x_limits))
709     ax.legend()
710
711     text = _mixture_text(log_mixture)
712     ax.text(0.02, 0.95, text, transform=ax.transAxes, va="top", fontsize=9)
713     _style_large_plot(fig, ax)
714     fig.savefig(path, dpi=160)
715     plt.close(fig)
716     # return path
717
718
719 def plot_log_actual_error_fit(
720     results_path: Path,
721     output_dir: Path,
722     manifest_path: Path | None = None,
723     bins: int = 30,
724     log_base: float = math.e,
725 ) -> Path | None:
726     plt = _import_pyplot()
727     rows = _read_cluster_runtime_rows(results_path)
728     errors = _cluster_actual_errors_from_manifest(rows, manifest_path)
729     if len(errors) < 2:
730         return None
731
732     mixture = _rescale_lognormal_mixture(
733         fit_lognormal_mixture(errors, component_count=2), log_base
734     )
735     log_errors = _log_values(errors, log_base)
736     output_dir.mkdir(parents=True, exist_ok=True)
737     path = output_dir / "log_actual_error_fit.png"
738     x_limits = _main_x_limits(log_errors)
739     histogram_bins = _compact_hist_bins(errors, bins)
740
741     fig, ax = plt.subplots(figsize=(12, 6))
742     ax.hist(
743         log_errors,
744         bins=histogram_bins,
745         range=x_limits,
746         color="#f58518",
747         edgecolor="white",
748         alpha=0.85,
749         label="логарифм фактической ошибки",
750     )
751     _plot_normal_mixture_pdf(ax, mixture, log_errors, histogram_bins, x_limits=x_limits)
752     ax.set_title("Фактическое распределение логарифма ошибки прогноза времени")
753     ax.set_xlabel(
754         _log_axis_label(log_base, "расчетный прогноз из manifest - фактическое время")
755     )
756     ax.set_ylabel("количество задач")
757     ax.set_xlim(*_padded_x_limits(x_limits))
758     ax.legend()
759
760     text = _mixture_text(mixture)
761     ax.text(0.02, 0.95, text, transform=ax.transAxes, va="top", fontsize=9)
762     _style_large_plot(fig, ax)
763     fig.savefig(path, dpi=160)
764     plt.close(fig)
765     return path
766
767
768 def plot_cluster_runtime_spread(
769     results_path: Path, output_dir: Path, max_group_size: int
770 ) -> Path | None:
771     plt = _import_pyplot()
772     rows = _read_cluster_runtime_rows(results_path)
773
774     if not rows:
775         return None
776
777     print(f"rows={len(rows)}")
778
779     indexes, planned, actual, overhead = _cluster_runtime_series(rows)

```

```

780
781 output_dir.mkdir(parents=True, exist_ok=True)
782 path = output_dir / "actual_runtime_spread.png"
783
784 fig, axes = plt.subplots(2, 2, figsize=(14, 9))
785 flat_axes = axes.flatten()
786
787 lower = min(min(planned), min(actual))
788 upper = max(max(planned), max(actual))
789 flat_axes[0].scatter(planned, actual, s=26, alpha=0.65, color="#4c78a8")
790 flat_axes[0].plot(
791     [lower, upper],
792     [lower, upper],
793     color="#d62728",
794     linewidth=1.5,
795     linestyle="--",
796     label="факт = план",
797 )
798 flat_axes[0].set_title("Плановое и фактическое время")
799 flat_axes[0].set_xlabel("запланированный sleep, секунд")
800 flat_axes[0].set_ylabel("фактическое время Slurm, секунд")
801 flat_axes[0].legend()
802
803 flat_axes[1].hist(
804     overhead,
805     bins=min(14, max(5, len(set(overhead)))),
806     color="#f58518",
807     edgecolor="white",
808 )
809 overhead_xlim = _padded_x_limits((min(overhead), max(overhead)), padding_ratio=0.08)
810 flat_axes[1].set_xlim(*overhead_xlim)
811 if overhead_xlim[0] <= 3.0 <= overhead_xlim[1]:
812     flat_axes[1].axvline(
813         3.0, color="#d62728", linestyle="--", linewidth=1.5, label="порог 3%"
814     )
815 flat_axes[1].set_title("Распределение отклонения")
816 flat_axes[1].set_xlabel("(факт - sleep) / sleep, %")
817 flat_axes[1].set_ylabel("количество задач")
818 if flat_axes[1].get_legend_handles_labels()[0]:
819     flat_axes[1].legend()
820
821 group_size = max(1, math.ceil(len(indexes) / max_group_size))
822 grouped_indexes: list[float] = []
823 grouped_planned: list[float] = []
824 grouped_actual: list[float] = []
825 grouped_overhead: list[float] = []
826 for start in range(0, len(indexes), group_size):
827     end = start + group_size
828     grouped_indexes.append(statistics.mean(indexes[start:end]))
829     grouped_planned.append(statistics.median(planned[start:end]))
830     grouped_actual.append(statistics.median(actual[start:end]))
831     grouped_overhead.append(statistics.median(overhead[start:end]))
832
833 flat_axes[2].plot(
834     grouped_indexes,
835     grouped_planned,
836     marker="o",
837     linewidth=1.8,
838     label="медиана planned sleep",
839 )
840 flat_axes[2].plot(
841     grouped_indexes,
842     grouped_actual,
843     marker="o",
844     linewidth=1.8,
845     label="медиана факта",
846 )
847 flat_axes[2].set_title(f"Медианы по группам задач, группа до {group_size}")
848 flat_axes[2].set_xlabel("номер задачи")
849 flat_axes[2].set_ylabel("секунд")
850 flat_axes[2].legend()
851
852 flat_axes[3].plot(
853     grouped_indexes,
854     grouped_overhead,
855     marker="o",
856     linewidth=1.8,
857     color="#54a24b",

```

```

858         label="медиана отклонения",
859     )
860     flat_axes[3].axhline(
861         3.0, color="#d62728", linestyle="--", linewidth=1.5, label="порог 3%"
862     )
863     flat_axes[3].set_title("Медианное отклонение по группам")
864     flat_axes[3].set_xlabel("номер задачи")
865     flat_axes[3].set_ylabel("(факт - sleep) / sleep, %")
866     flat_axes[3].legend()
867
868     fig.tight_layout()
869     fig.savefig(path, dpi=160)
870     plt.close(fig)
871     return path
872
873
874 def plot_cluster_planned_actual_by_job(
875     results_path: Path,
876     output_dir: Path,
877     max_group_size: int,
878 ) -> Path | None:
879     plt = _import_pyplot()
880     rows = _read_cluster_runtime_rows(results_path)
881     if not rows:
882         return None
883
884     indexes, planned, actual, _ = _cluster_runtime_series(rows)
885     output_dir.mkdir(parents=True, exist_ok=True)
886     path = output_dir / "planned_vs_actual_by_job.png"
887
888     fig, ax = plt.subplots(figsize=(12, 5))
889     ax.scatter(indexes, planned, s=18, alpha=0.55, color="#4c78a8")
890     ax.scatter(indexes, actual, s=18, alpha=0.55, color="#f58518")
891
892     group_size = max(1, math.ceil(len(indexes) / max_group_size))
893     grouped_indexes: list[float] = []
894     grouped_planned: list[float] = []
895     grouped_actual: list[float] = []
896     for start in range(0, len(indexes), group_size):
897         end = start + group_size
898         grouped_indexes.append(statistics.mean(indexes[start:end]))
899         grouped_planned.append(statistics.median(planned[start:end]))
900         grouped_actual.append(statistics.median(actual[start:end]))
901
902     ax.plot(
903         grouped_indexes,
904         grouped_planned,
905         color="#4c78a8",
906         linewidth=2.2,
907         marker="o",
908         label="запланированный sleep",
909     )
910     ax.plot(
911         grouped_indexes,
912         grouped_actual,
913         color="#f58518",
914         linewidth=2.2,
915         marker="o",
916         label="фактическое время Slurm",
917     )
918     ax.set_title("Плановое и фактическое время по задачам")
919     ax.set_xlabel("номер задачи")
920     ax.set_ylabel("секунд")
921     ax.legend()
922     fig.tight_layout()
923     fig.savefig(path, dpi=160)
924     plt.close(fig)
925     return path
926
927
928 def plot_cluster_overhead_by_job(
929     results_path: Path, output_dir: Path, max_group_size: int
930 ) -> Path | None:
931     plt = _import_pyplot()
932     rows = _read_cluster_runtime_rows(results_path)
933     if not rows:
934         return None
935

```

```

936 indexes, _, _, overhead = _cluster_runtime_series(rows)
937 output_dir.mkdir(parents=True, exist_ok=True)
938 path = output_dir / "actual_overhead_by_job.png"
939
940 fig, ax = plt.subplots(figsize=(12, 5))
941 ax.scatter(indexes, overhead, s=18, alpha=0.55, color="#54a24b")
942
943 group_size = max(1, math.ceil(len(indexes) / max_group_size))
944 grouped_indexes: list[float] = []
945 grouped_overhead: list[float] = []
946 for start in range(0, len(indexes), group_size):
947     end = start + group_size
948     grouped_indexes.append(statistics.mean(indexes[start:end]))
949     grouped_overhead.append(statistics.median(overhead[start:end]))
950
951 ax.plot(
952     grouped_indexes,
953     grouped_overhead,
954     color="#54a24b",
955     linewidth=2.2,
956     marker="o",
957     label="медиана по группам",
958 )
959 ax.axhline(3.0, color="#d62728", linestyle="--", linewidth=1.5, label="порог 3%")
960 ax.set_title("Отклонение фактического времени по задачам")
961 ax.set_xlabel("номер задачи")
962 ax.set_ylabel("(факт - sleep) / sleep, %")
963 ax.legend()
964 fig.tight_layout()
965 fig.savefig(path, dpi=160)
966 plt.close(fig)
967 return path
968
969
970 def build_graphs(
971     jobs: list[SourceJob],
972     max_group_size: int,
973     output_dir: Path,
974     manifest_path: Path | None,
975     log_path: Path | None,
976     cluster_results_path: Path | None = None,
977     log_base: float = math.e,
978 ) -> list[Path]:
979     clean_graphs_dir(output_dir)
980     paths = [
981         plot_source_summary(jobs, output_dir),
982         plot_runtime_fit(jobs, output_dir),
983         plot_forecast_error_fit(
984             jobs,
985             output_dir,
986             manifest_path=manifest_path,
987             cluster_results_path=cluster_results_path,
988         ),
989         plot_log_forecast_error_fit(jobs, output_dir, log_base=log_base),
990     ]
991     if manifest_path and manifest_path.exists():
992         paths.append(plot_generated_vs_source(jobs, manifest_path, output_dir))
993     if manifest_path and log_path:
994         overhead = plot_execution_overhead(manifest_path, log_path, output_dir)
995         if overhead:
996             paths.append(overhead)
997     if cluster_results_path:
998         planned_actual = plot_cluster_planned_actual_by_job(
999             cluster_results_path, output_dir, max_group_size
1000         )
1001         if planned_actual:
1002             paths.append(planned_actual)
1003         overhead_by_job = plot_cluster_overhead_by_job(
1004             cluster_results_path, output_dir, max_group_size
1005         )
1006         if overhead_by_job:
1007             paths.append(overhead_by_job)
1008         actual_error = plot_actual_error_fit(
1009             cluster_results_path,
1010             output_dir,
1011             manifest_path=manifest_path,
1012             log_base=log_base,
1013         )

```

```

1014         if actual_error:
1015             paths.append(actual_error)
1016         log_actual_error = plot_log_actual_error_fit(
1017             cluster_results_path,
1018             output_dir,
1019             manifest_path=manifest_path,
1020             log_base=log_base,
1021         )
1022         if log_actual_error:
1023             paths.append(log_actual_error)
1024     return paths

```

Файл src/analyze/main.py

```

1  from __future__ import annotations
2
3  import argparse
4  import math
5  from pathlib import Path
6
7  from src.analyze.analyze import DEFAULT_DATASET, print_summary, read_jobs
8  from src.analyze.graphs import build_graphs
9
10
11  DEFAULT_GRAPHS_DIR = Path("generated/graphs")
12
13
14  def positive_float(value: str) -> float:
15      result = float(value)
16      if result <= 0:
17          raise argparse.ArgumentTypeError("value must be positive")
18      return result
19
20
21  def positive_int(value: str) -> int:
22      result = int(value)
23      if result <= 0:
24          raise argparse.ArgumentTypeError("value must be positive")
25      return result
26
27
28  def log_base(value: str) -> float:
29      if value.lower() in {"e", "natural", "ln"}:
30          return math.e
31      result = positive_float(value)
32      if result == 1:
33          raise argparse.ArgumentTypeError("log base must not be equal to 1")
34      return result
35
36
37  def parse_args() -> argparse.Namespace:
38      parser = argparse.ArgumentParser(description="Analyze filtered SCC accounting data.")
39      parser.add_argument("--dataset", type=Path, default=DEFAULT_DATASET)
40      parser.add_argument("--graphs-dir", type=Path, default=DEFAULT_GRAPHS_DIR)
41      parser.add_argument("--manifest", type=Path,
42          ↪ default=Path("generated/slurm_jobs/manifest.csv"))
43      parser.add_argument("--execution-log", type=Path,
44          ↪ default=Path("generated/logs/generated-jobs.log"))
45      parser.add_argument("--cluster-results", type=Path)
46      parser.add_argument(
47          "--log-base",
48          type=log_base,
49          default=math.e,
50          help="Logarithm base for log-scale graphs. Use 10 for decimal logarithm or e for
51          ↪ natural logarithm.",
52      )
53      parser.add_argument(
54          "--max-group-size",
55          type=positive_int,
56          default=50,
57          help="Maximum group size for smoothed cluster result graphs.",
58      )
59      parser.add_argument("--no-graphs", action="store_true")
60      return parser.parse_args()

```

```

60 def main() -> None:
61     args = parse_args()
62     jobs = read_jobs(args.dataset)
63     if not jobs:
64         raise SystemExit(f"no jobs read from {args.dataset}")
65
66     print_summary(jobs)
67     if args.no_graphs:
68         return
69
70     paths = build_graphs(
71         jobs=jobs,
72         output_dir=args.graphs_dir,
73         manifest_path=args.manifest,
74         log_path=args.execution_log,
75         cluster_results_path=args.cluster_results,
76         max_group_size=args.max_group_size,
77         log_base=args.log_base,
78     )
79     for path in paths:
80         print(f"graph={path}")
81
82
83 if __name__ == "__main__":
84     main()

```

Файл src/pipeline/cluster.py

```

1  from __future__ import annotations
2
3  import csv
4  import datetime as dt
5  import shlex
6  import subprocess
7  import time
8  from dataclasses import dataclass
9  from pathlib import Path
10
11
12 RESULT_FIELDS = [
13     "script",
14     "slurm_job_id",
15     "source_job_id",
16     "planned_sleep_seconds",
17     "requested_slurm_time",
18     "submitted_observed_at",
19     "completed_observed_at",
20     "slurm_submit_time",
21     "slurm_start_time",
22     "slurm_end_time",
23     "slurm_state",
24     "node_list",
25     "queue_wait_seconds",
26     "actual_slurm_runtime_seconds",
27     "observed_wall_seconds",
28     "overhead_vs_sleep_percent",
29 ]
30
31
32 @dataclass(frozen=True)
33 class RemoteRun:
34     local_run_dir: Path
35     remote_run_dir: str
36     manifest_path: Path
37
38
39 def shell_quote(value: str | Path) -> str:
40     raw = str(value)
41     if raw == "~":
42         return raw
43     if raw.startswith("~/"):
44         return "~/ " + shlex.quote(raw[2:])
45     return shlex.quote(raw)
46
47
48 def run_command(command: list[str]) -> str:

```

```

49     completed = subprocess.run(command, check=True, text=True, capture_output=True)
50     return completed.stdout.strip()
51
52
53 def ssh(host: str, command: str) -> str:
54     return run_command(["ssh", host, command])
55
56
57 def try_ssh(host: str, command: str) -> str:
58     try:
59         return ssh(host, command)
60     except subprocess.CalledProcessError:
61         return ""
62
63
64 def scp_from(source: str, target: Path) -> None:
65     subprocess.run(["scp", source, str(target)], check=True)
66
67
68 def timestamp() -> str:
69     return dt.datetime.now().astimezone().isoformat(timespec="seconds")
70
71
72 def parse_slurm_fields(raw: str) -> dict[str, str]:
73     result: dict[str, str] = {}
74     for item in raw.split():
75         if "=" not in item:
76             continue
77         key, value = item.split("=", 1)
78         result[key] = value
79     return result
80
81
82 def parse_datetime(value: str) -> dt.datetime | None:
83     if not value or value in {"Unknown", "N/A", "None"}:
84         return None
85     return dt.datetime.fromisoformat(value)
86
87
88 def seconds_between(start: dt.datetime | None, end: dt.datetime | None) -> float | None:
89     if start is None or end is None:
90         return None
91     return (end - start).total_seconds()
92
93
94 def read_manifest(path: Path) -> dict[str, dict[str, str]]:
95     with path.open(newline="", encoding="utf-8") as input_file:
96         return {row["script"]: row for row in csv.DictReader(input_file)}
97
98
99 def run_remote_generator(
100     host: str,
101     generator_dir: str,
102     generator_command: str,
103     results_root: Path,
104 ) -> RemoteRun:
105     run_id = "run_" + dt.datetime.now().strftime("%Y%m%d_%H%M%S")
106     local_run_dir = results_root / run_id
107     local_run_dir.mkdir(parents=True, exist_ok=True)
108
109     remote_run_dir = ssh(
110         host,
111         "cd "
112         f"{shell_quote(generator_dir)} && "
113         f"{shell_quote(generator_command)} --run-id {shell_quote(run_id)}",
114     )
115     if not remote_run_dir:
116         raise SystemExit("remote generator did not print remote run directory")
117
118     manifest_path = local_run_dir / "manifest.csv"
119     scp_from(f"{host}:{remote_run_dir}/manifest.csv", manifest_path)
120     return RemoteRun(
121         local_run_dir=local_run_dir,
122         remote_run_dir=remote_run_dir,
123         manifest_path=manifest_path,
124     )
125
126

```

```

127 def copy_existing_manifest(
128     host: str,
129     remote_run_dir: str,
130     results_root: Path,
131 ) -> RemoteRun:
132     run_id = Path(remote_run_dir.rstrip("/")).name
133     local_run_dir = results_root / run_id
134     local_run_dir.mkdir(parents=True, exist_ok=True)
135     manifest_path = local_run_dir / "manifest.csv"
136     scp_from(f"{host}:{remote_run_dir}/manifest.csv", manifest_path)
137     return RemoteRun(
138         local_run_dir=local_run_dir,
139         remote_run_dir=remote_run_dir,
140         manifest_path=manifest_path,
141     )
142
143
144 def build_result_row(
145     host: str,
146     submitted_row: dict[str, str],
147     manifest_row: dict[str, str],
148 ) -> dict[str, str | int]:
149     completed_at = timestamp()
150     slurm_raw = try_ssh(host, f"scontrol show job {submitted_row['slurm_job_id']} -o")
151     slurm = parse_slurm_fields(slurm_raw)
152
153     submit_time = parse_datetime(slurm.get("SubmitTime", ""))
154     start_time = parse_datetime(slurm.get("StartTime", ""))
155     end_time = parse_datetime(slurm.get("EndTime", ""))
156     planned_sleep = int(manifest_row["scaled_sleep_seconds"])
157     actual_runtime = seconds_between(start_time, end_time)
158     queue_wait = seconds_between(submit_time, start_time)
159     observed_wall = seconds_between(
160         dt.datetime.fromisoformat(submitted_row["submitted_observed_at"]),
161         dt.datetime.fromisoformat(completed_at),
162     )
163     overhead = None
164     if actual_runtime is not None:
165         overhead = (actual_runtime - planned_sleep) / planned_sleep * 100.0
166
167     return {
168         "script": submitted_row["script"],
169         "slurm_job_id": submitted_row["slurm_job_id"],
170         "source_job_id": manifest_row["source_job_id"],
171         "planned_sleep_seconds": planned_sleep,
172         "requested_slurm_time": manifest_row["slurm_time"],
173         "submitted_observed_at": submitted_row["submitted_observed_at"],
174         "completed_observed_at": completed_at,
175         "slurm_submit_time": slurm.get("SubmitTime", ""),
176         "slurm_start_time": slurm.get("StartTime", ""),
177         "slurm_end_time": slurm.get("EndTime", ""),
178         "slurm_state": slurm.get("JobState", "UNKNOWN"),
179         "node_list": slurm.get("NodeList", ""),
180         "queue_wait_seconds": f"{queue_wait:.3f}" if queue_wait is not None else "",
181         "actual_slurm_runtime_seconds": (
182             f"{actual_runtime:.3f}" if actual_runtime is not None else ""
183         ),
184         "observed_wall_seconds": (
185             f"{observed_wall:.3f}" if observed_wall is not None else ""
186         ),
187         "overhead_vs_sleep_percent": (
188             f"{overhead:.3f}" if overhead is not None else ""
189         ),
190     }
191
192
193 def collect_results_as_jobs_finish(
194     host: str,
195     submitted: list[dict[str, str]],
196     manifest: dict[str, dict[str, str]],
197     result_path: Path,
198     poll_interval: float,
199 ) -> None:
200     pending = {row["slurm_job_id"]: row for row in submitted}
201     with result_path.open("w", newline="", encoding="utf-8") as output_file:
202         writer = csv.DictWriter(output_file, fieldnames=RESULT_FIELDS)
203         writer.writeheader()
204

```

```

205     while pending:
206         ids = ",".join(sorted(pending))
207         raw = ssh(host, f"squeue -h -j {ids} -o '%i' || true")
208         active = {line.strip() for line in raw.splitlines() if line.strip()}
209         finished = sorted(set(pending) - active, key=int)
210
211         for job_id in finished:
212             submitted_row = pending.pop(job_id)
213             writer.writerow(
214                 build_result_row(
215                     host=host,
216                     submitted_row=submitted_row,
217                     manifest_row=manifest[submitted_row["script"]],
218                 )
219             )
220             output_file.flush()
221
222         if pending:
223             time.sleep(poll_interval)
224
225
226 def submit_remote_run(
227     host: str,
228     remote_run_dir: str,
229     local_run_dir: Path,
230     manifest_path: Path,
231     poll_interval: float,
232 ) -> Path:
233     manifest = read_manifest(manifest_path)
234     submitted: list[dict[str, str]] = []
235     for script_name in sorted(manifest):
236         submitted_at = timestamp()
237         job_id = ssh(
238             host,
239             "cd "
240             f"{shell_quote(remote_run_dir)} && "
241             f"sbatch --parsable {shell_quote(Path('slurm_jobs') / script_name)}",
242         )
243         submitted.append(
244             {
245                 "script": script_name,
246                 "slurm_job_id": job_id,
247                 "submitted_observed_at": submitted_at,
248             }
249         )
250
251     result_path = local_run_dir / "results.csv"
252     collect_results_as_jobs_finish(
253         host=host,
254         submitted=submitted,
255         manifest=manifest,
256         result_path=result_path,
257         poll_interval=poll_interval,
258     )
259     return result_path

```

Файл src/pipeline/main.py

```

1  from __future__ import annotations
2
3  import argparse
4  import json
5  import math
6  from pathlib import Path
7  from typing import Any
8
9  from src.analyze.analyze import print_summary
10 from src.analyze.graphs import build_graphs
11 from src.core.jobs import read_jobs
12 from src.pipeline.cluster import (
13     copy_existing_manifest,
14     run_remote_generator,
15     submit_remote_run,
16 )
17
18 DEFAULT_CONFIG = Path("config/pipeline.json")

```

```

19
20
21 def positive_int(value: str) -> int:
22     parsed = int(value)
23     if parsed <= 0:
24         raise argparse.ArgumentTypeError("value must be positive")
25     return parsed
26
27
28 def log_base(value: str) -> float:
29     if value.lower() in {"e", "natural", "ln"}:
30         return math.e
31     parsed = float(value)
32     if parsed <= 0 or parsed == 1:
33         raise argparse.ArgumentTypeError("log base must be positive and not equal to 1")
34     return parsed
35
36
37 def load_config(path: Path) -> dict[str, Any]:
38     with path.open(encoding="utf-8") as input_file:
39         return json.load(input_file)
40
41
42 def path_from_config(config: dict[str, Any], key: str) -> Path:
43     value = config.get(key)
44     if not value:
45         raise SystemExit(f"missing required config key: {key}")
46     return Path(value)
47
48
49 def component_args(option: str, components: list[dict[str, Any]]) -> str:
50     return "".join(
51         f" {option} "
52         f"{{float(component['mu'])}}, {{float(component['sigma'])}}, {{float(component['weight'])}} "
53         for component in components
54     )
55
56
57 def normal_component_args(option: str, components: list[dict[str, Any]]) -> str:
58     return "".join(
59         f" {option} "
60
61         ↪ f"{{float(component['mean'])}}, {{float(component['sigma'])}}, {{float(component['weight'])}} "
62         for component in components
63     )
64
65 def parse_args() -> argparse.Namespace:
66     parser = argparse.ArgumentParser(
67         description="Run generation, optional Slurm execution, and analysis synchronously."
68     )
69     parser.add_argument("--config", type=Path, default=DEFAULT_CONFIG)
70     parser.add_argument(
71         "--analyze-only",
72         action="store_true",
73         help="Build graphs from existing local manifest/results without touching Slurm.",
74     )
75     parser.add_argument(
76         "--no-graphs",
77         action="store_true",
78         help="Skip graph generation after analysis.",
79     )
80     parser.add_argument("--count", type=positive_int, help="Override configured count.")
81     parser.add_argument(
82         "--log-base",
83         type=log_base,
84         help="Override logarithm base for log-scale graphs. Use 10 for decimal logarithm or  

85         ↪ e for natural logarithm.",
86     )
87     parser.add_argument("--remote-run-dir", help="Use an existing generated run on VM.")
88     return parser.parse_args()
89
90 def main() -> None:
91     args = parse_args()
92     config = load_config(args.config)
93
94     dataset_path = path_from_config(config, "dataset")

```

```

95 generator_config = config.get("remote_generator", {})
96 cluster_config = config.get("cluster", {})
97 analyze_config = config.get("analyze", {})
98
99 count = (
100     args.count if args.count is not None else int(generator_config.get("count", 20))
101 )
102 max_group_size = int(analyze_config.get("max_group_size", 50))
103 graph_log_base = (
104     args.log_base
105     if args.log_base is not None
106     else log_base(str(analyze_config.get("log_base", math.e)))
107 )
108 graphs_dir = Path(analyze_config.get("graphs_dir", "generated/graphs"))
109 results_root = Path(cluster_config.get("results_root", "generated/cluster_runs"))
110
111 jobs = read_jobs(dataset_path)
112 if not jobs:
113     raise SystemExit(f"no jobs read from {dataset_path}")
114
115 manifest_path = Path("generated/slurm_jobs/manifest.csv")
116 cluster_results_path: Path | None = None
117
118 if not args.analyze_only and bool(cluster_config.get("enabled", False)):
119     host = str(cluster_config.get("host", generator_config.get("host", "mgmt")))
120     remote_run_dir = (
121         args.remote_run_dir
122         or str(cluster_config.get("remote_run_dir") or "").strip()
123     )
124
125     if remote_run_dir:
126         print(f"stage=manifest host={host} remote_run_dir={remote_run_dir}")
127         remote_run = copy_existing_manifest(
128             host=host,
129             remote_run_dir=remote_run_dir,
130             results_root=results_root,
131         )
132     else:
133         generator_host = str(generator_config.get("host", host))
134         generator_dir = str(
135             generator_config.get("generator_dir", "~/slurm-generator")
136         )
137         runtime_components = generator_config.get(
138             "runtime_components",
139             [{"mean": 1701.49, "sigma": 435.21, "weight": 1.0}],
140         )
141         error_components = generator_config.get(
142             "error_components",
143             [{"mu": 9.8981, "sigma": 0.0371, "weight": 1.0}],
144         )
145         generator_command = (
146             str(generator_config.get("command", "python3 -m generate.main"))
147             + f" --count {count}"
148             + f" --sleep-scale {float(generator_config.get('sleep_scale', 0.01))}"
149             + f" --time-scale {float(generator_config.get('time_scale', 0.01))}"
150             + normal_component_args(
151                 "--runtime-normal-component", runtime_components
152             )
153             + component_args("--error-component", error_components)
154             + f" --seed {int(generator_config.get('seed', 50728))}"
155             + f" --partition {str(generator_config.get('partition', 'debug'))}"
156         )
157         print(
158             f"stage=generate host={generator_host} "
159             f"generator_dir={generator_dir} count={count}"
160         )
161         remote_run = run_remote_generator(
162             host=generator_host,
163             generator_dir=generator_dir,
164             generator_command=generator_command,
165             results_root=results_root,
166         )
167
168 manifest_path = remote_run.manifest_path
169 print(f"generated_manifest={manifest_path}")
170 print(f"remote_run_dir={remote_run.remote_run_dir}")
171

```

```

172     print(f"stage=cluster host={host}")
173     cluster_results_path = submit_remote_run(
174         host=host,
175         remote_run_dir=remote_run.remote_run_dir,
176         local_run_dir=remote_run.local_run_dir,
177         manifest_path=manifest_path,
178         poll_interval=float(cluster_config.get("poll_interval", 1.0)),
179     )
180 else:
181     print("stage=cluster skipped=true")
182     latest_results = sorted(results_root.glob("run_*/results.csv"))
183     if latest_results:
184         cluster_results_path = latest_results[-1]
185         manifest_path = cluster_results_path.parent / "manifest.csv"
186
187     print("stage=analyze")
188     print_summary(jobs)
189     if args.no_graphs:
190         return
191
192     graph_paths = build_graphs(
193         jobs=jobs,
194         output_dir=graphs_dir,
195         manifest_path=manifest_path,
196         log_path=None,
197         cluster_results_path=cluster_results_path,
198         max_group_size=max_group_size,
199         log_base=graph_log_base,
200     )
201     for path in graph_paths:
202         print(f"graph={path}")
203
204
205 if __name__ == "__main__":
206     main()

```